PSOC™ 4 CAPSENSE™ using PSOC™ 4000T and PSOC™ 4100T Plus training manual

About this document

This training manual covers the labs for the introduction to CAPSENSE™ on the PSOC™ 4000T and the PSOC™ 4100T Plus training.

Scope and purpose

The labs will cover objectives, project creation, tuning CAPSENSE™ sensors, updating the PSOC™ 4100T Plus for optimal power consumption, and takeaways from the labs.

Intended audience

The intended audiences for this document are design engineers, technicians, and developers of electronic systems.

Table of contents

Table of contents

1 Introduction

This manual provides instructions to create, configure, and build the **PSOC™ 4100T Plus MSCLP Low Power CSD Button** project to optimize the capacitive performance and reduce the power consumption.

2 Required development tools and prerequisites

2.1 Tools

1. ModusToolbox™ software v3.5 or later (tested with v3.5)
2. ModusToolbox™ CAPSENSE™ and Multi-Sense Pack
3. An oscilloscope and DMM will be used for CAPSENSE™ tuning and power measurements.
4. CY8CPROTO-041TP, the PSOC™ 4100T Plus Prototyping Kit

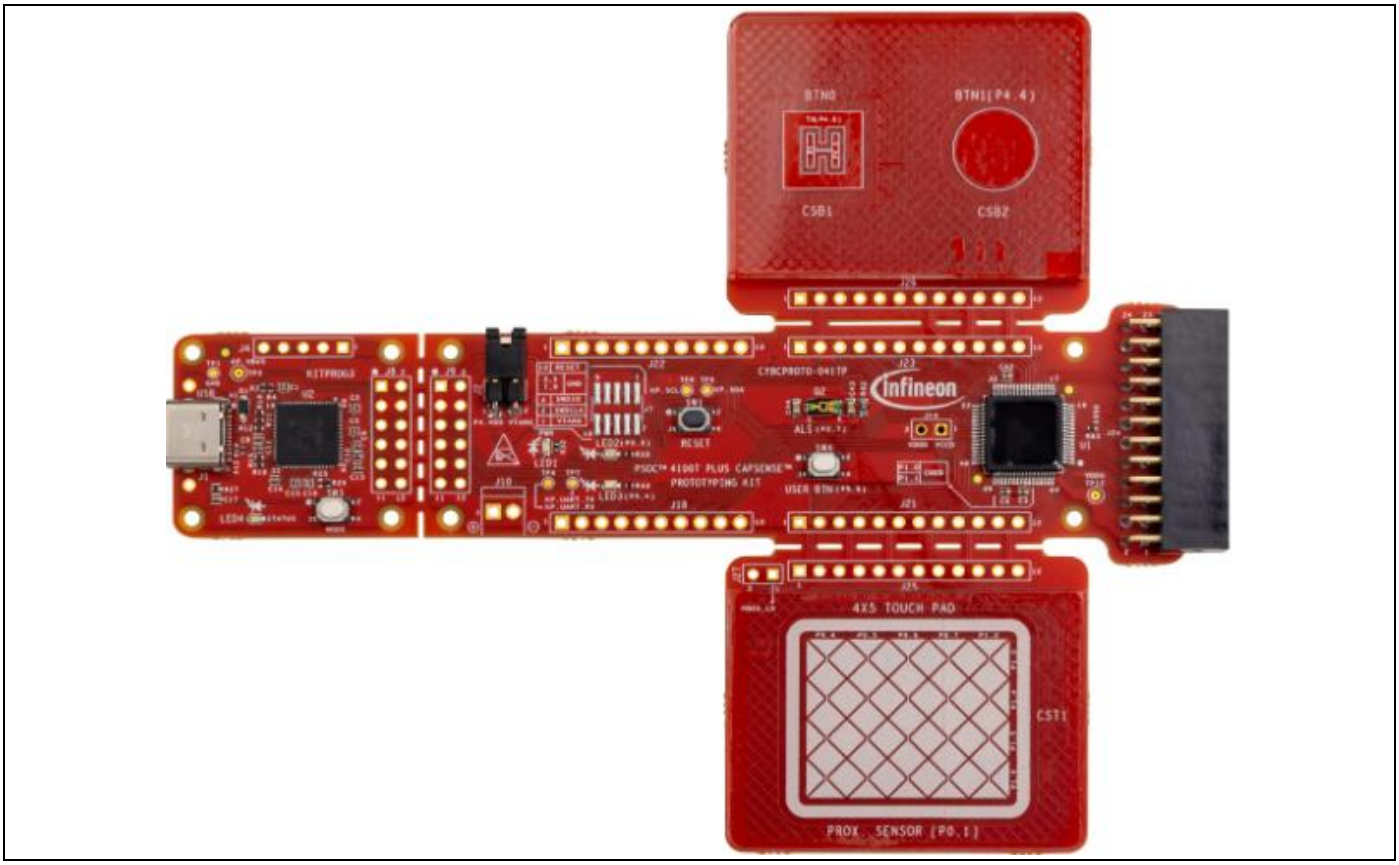

Figure 1 PSOC™ 4100T Plus Prototyping Kit

2.2 Prerequisites

1. Introduction to PSOC™ 4100T Plus Training
2. Install ModusToolbox™ software v3.5 (Eclipse IDE for ModusToolbox™ 3.4 is included)
3. Install the ModusToolbox™ CAPSENSE™ and Multi-Sense Pack

3 CAPSENSE™ performance tuning

3.1 Objective

The objective of this exercise is to learn how to go through the CAPSENSE™ performance tuning process. The MSCLP Low Power CSD Button example project will have the device configured for optimal capacitive sensor tuning, but tweaks can be made to show how each setting impacts the sensitivity of the sensors.

3.2 Description

When tuning a capacitive sensor, it is recommended to follow the CAPSENSE™ performance tuning process. There are five stages in this process:

1. Set the initial hardware parameters
2. Set the sense clock frequency
3. Set the Capacitive DAC (CDAC) tuning mode
4. Fine tune for the required SNR, power, and refresh rate
5. Tune the touch threshold parameters

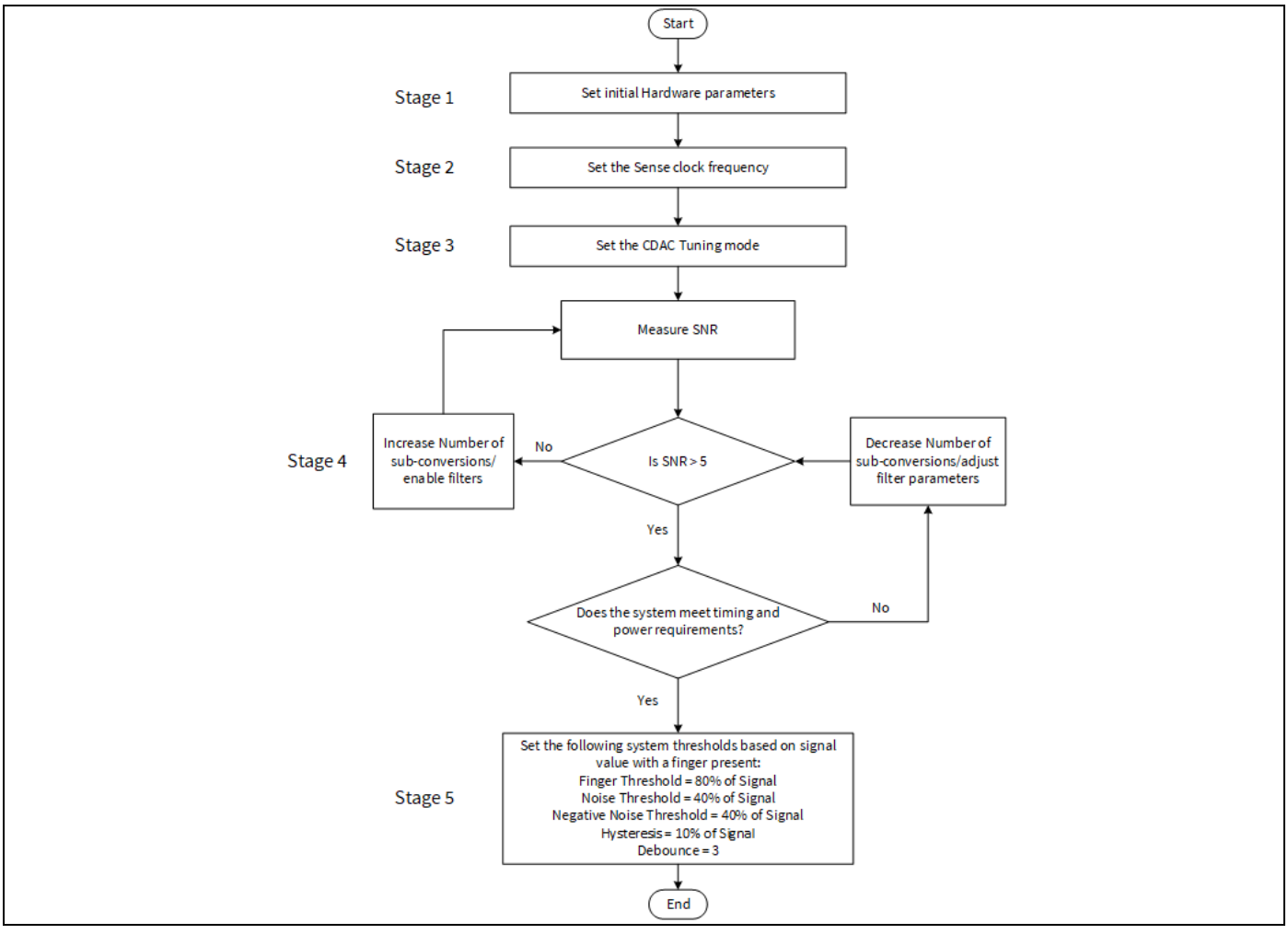

Figure 2 CAPSENSE™ performance tuning flowchart

3.3 Performance tuning steps

3.3.1 Project creation

1. Before creating the first project in this training series we need to create our workspace in Modus™ Toolbox. Open **ModusToolbox™** from the **Windows Start** menu.

   Note: If you have not installed ModusToolbox™, see the Required development tools section.

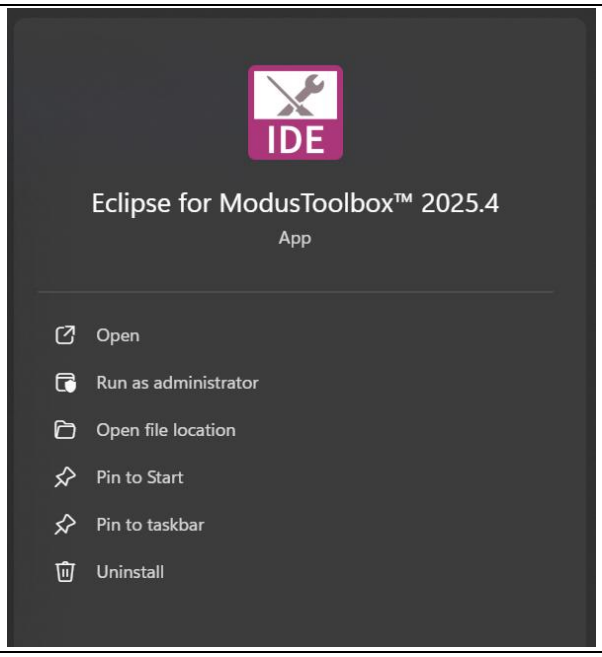

Figure 3 Running Eclipse IDE from the Windows 11 Search Bar

CAPSENSE™ performance tuning

2. **Next**, choose a **workspace directory** for your project. After selecting a suitable directory, click the Launch button.

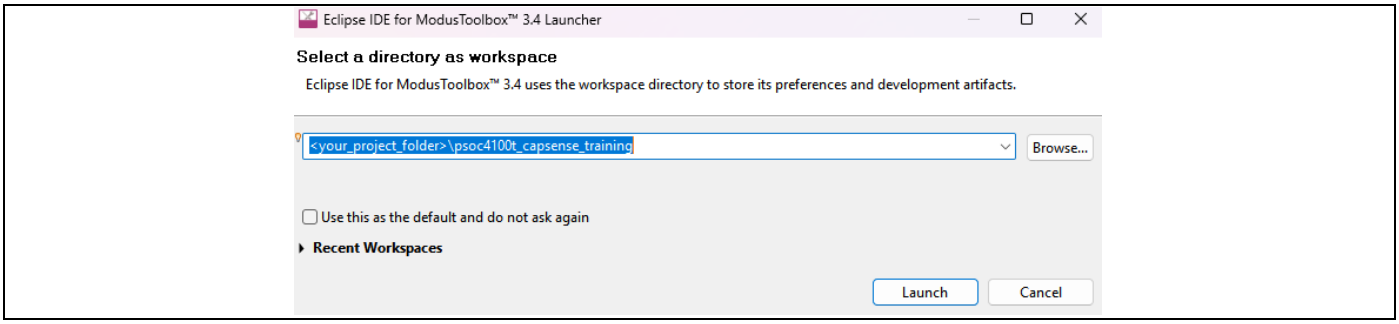

Figure 4 Launching the Eclipse workspace

3. Clicking Launch opens a new ModusToolbox™ workspace as shown below.

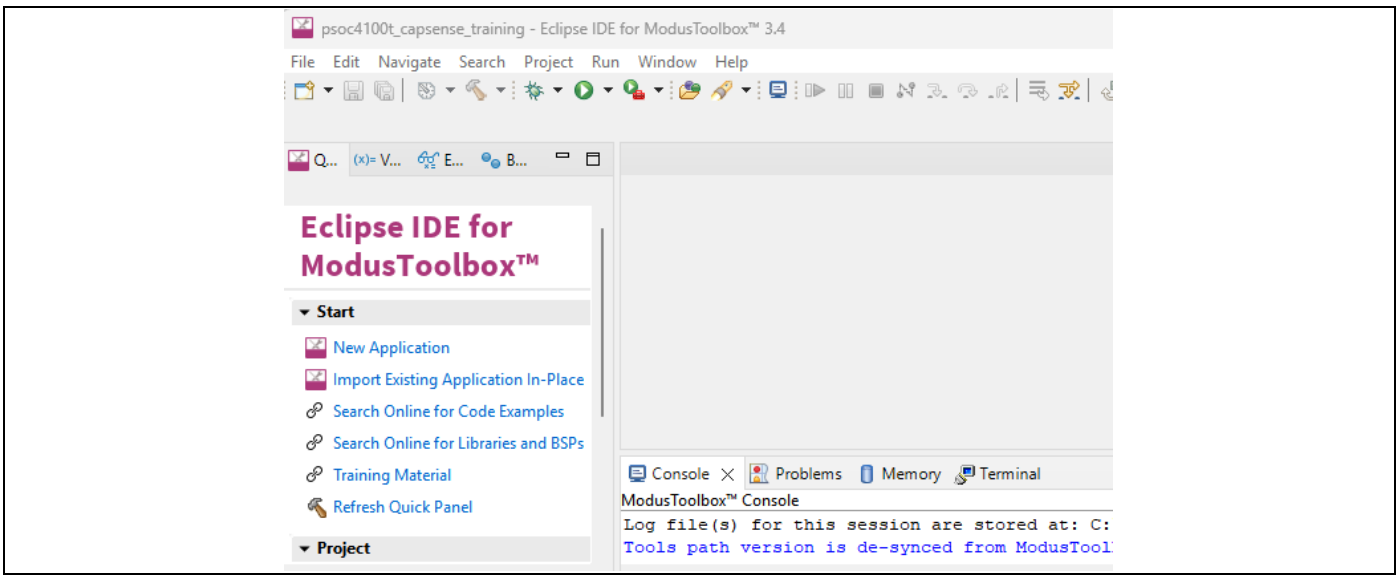

Figure 5 Eclipse IDE after ModusToolbox™ Launch

4. Now select **New Application** from the Quick Panel. Alternatively, go to **File > New > ModusToolbox™ Application**.

5. Clicking New Application opens the **Project Creator Tool** and prompts you to choose a **BSP** (Board Support Package).

   BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.

CAPSENSE™ performance tuning

6. Once **Project Creator** launches, Select the **CY8PROTO-041TP** BSP under the PSOC™ 4 BSPs. Then click **Next**.

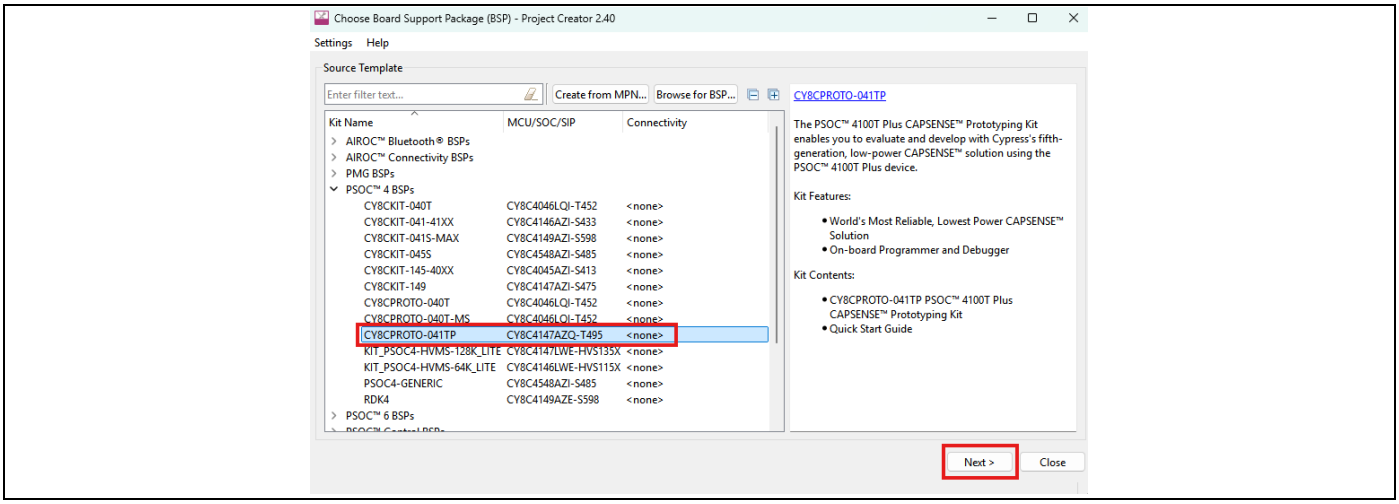

Figure 6 Project Creator CY8PROTO-041TP BSP Selection

7. Clicking **Next** takes you to the next menu of the Project Creator which asks you to select an application. Select the **MSCLP Low Power CSD Button** under the Sensing section. You can do this by selecting the **checkbox** near the application. Click **Create**.

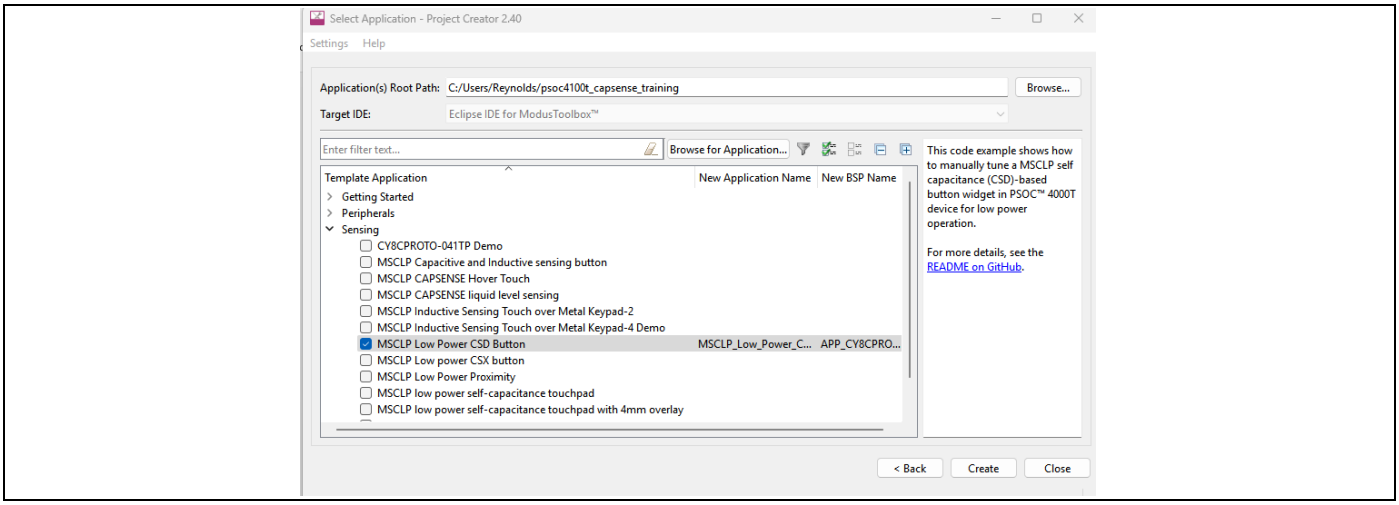

Figure 7 Project Creator example code project creation

8. When you click **Create**, a new project is created, and you can see all the files in the Project Explorer window.

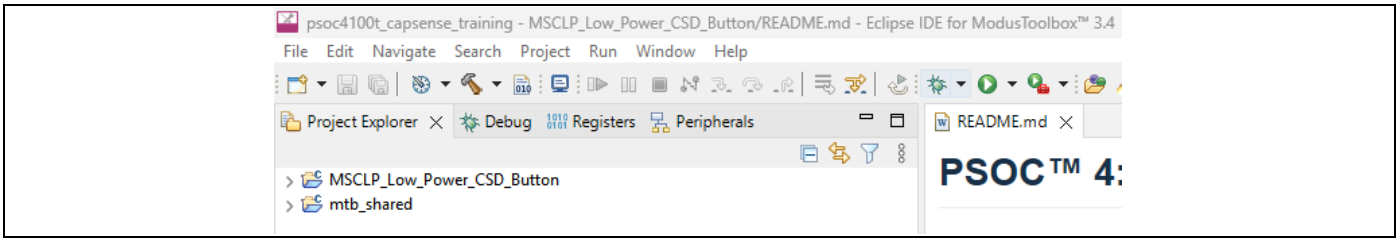

Figure 8 Eclipse IDE after project creation

CAPSENSE™ performance tuning

3.3.2 Stage 1 - Set the initial hardware parameters

The initial hardware parameters include:

- CAPSENSE™ IMO clock frequency setting
- Modulator clock divider
- Number of initial sub-conversions
- CIC2 hardware filter
- Hardware and software IIR filter configuration
- Inactive sensor connection
- Shield configuration (mode and count)
- Raw count calibration level
- Sense clock configuration (divider and clock source)
- Number of sub-conversions
- Decimation rate
- GPIO configuration
- Sensor pins, CMOD pins, and shield pins
- Scan slots
- Initial touch and noise thresholds, debounce, and hysteresis

1. Click on the **MSCLP_Low_Power_CSD_Button** project in the Project Explorer and the quick panel will populate the application information, tools, and API documentation. Open the CAPSENSE™ configurator tool from the quick panel.

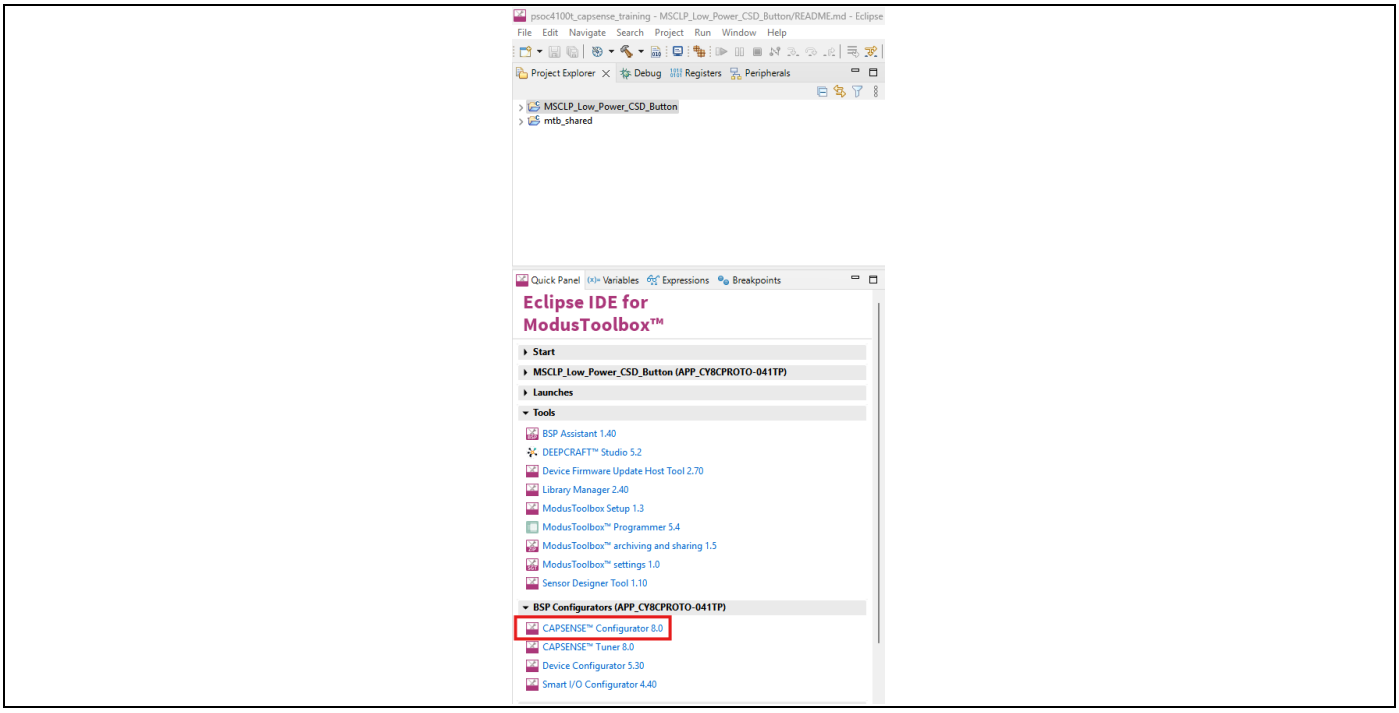

Figure 9 Selecting CAPSENSE™ Configurator from Eclipse IDE quick panel

CAPSENSE™ performance tuning

2. Open the **Advanced** → **General** tab to configure the initial CAPSENSE™ IMO clock frequency and divider as well as the initial filter settings.

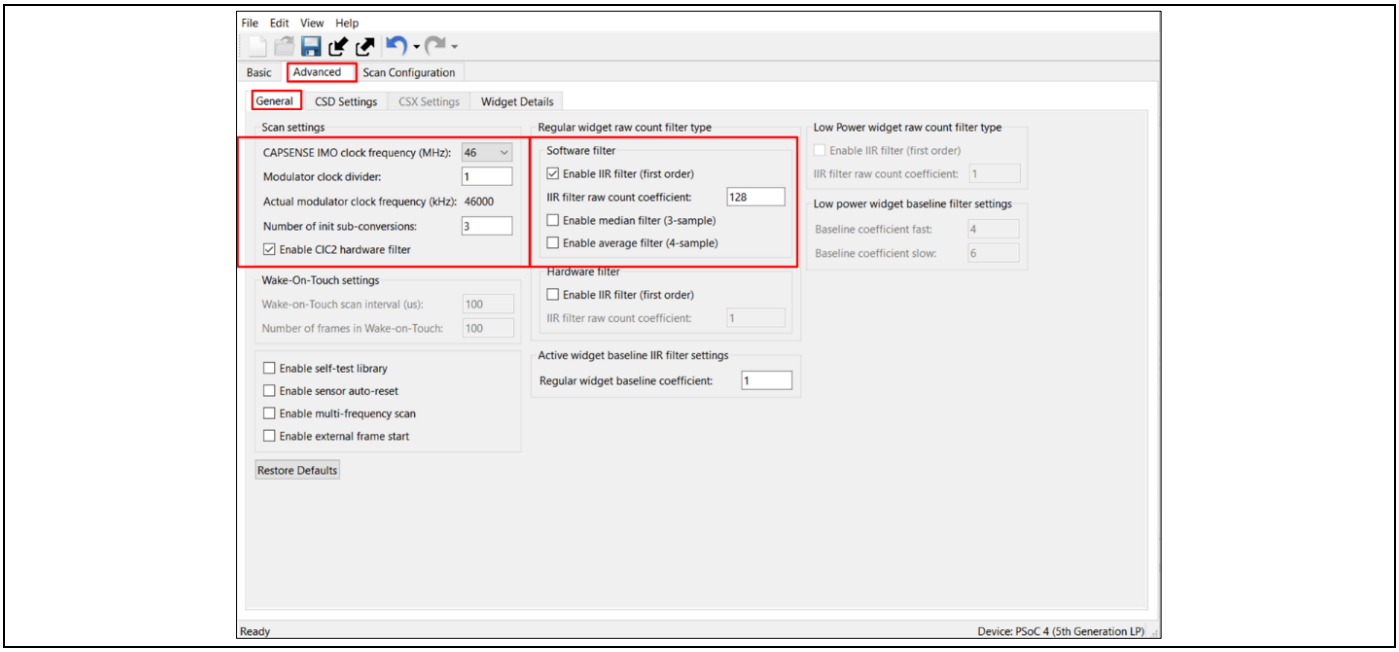

Figure 10 CAPSENSE™ Configurator general tab

Parameter	Setting	Comment
CAPSENSE™ IMO clock frequency (MHz)	Default: Max=46MHz	Frequency of clock used as source for the CAPSENSE™ peripheral
Modulator clock divider	Default: 1	Set to obtain the optimum modulator clock frequency
Number of init sub-conversions	Default: 3	To ensure proper initialization of CAPSENSE™ 3 init sub-conversion is required
Enable CIC2 HW filter	Default: Checked	Increases Signal and SNR.
Enable IIR filter (first order) (Software Filter)	Default: Checked	Reduces noise.
IIR filter raw count coefficient	Default: 128	A lower coefficient value results in lower noise, but slows down the response
Enable self-test library	Default: Uncheck	
Enable IIR filter (first order) (Hardware Filter)	Default: Unchecked	

Table 1 General CAPSENSE™ initial parameters

CAPSENSE™ performance tuning

3. Open the **Advanced** → **CSD Settings** tab to configure the initial shield settings.

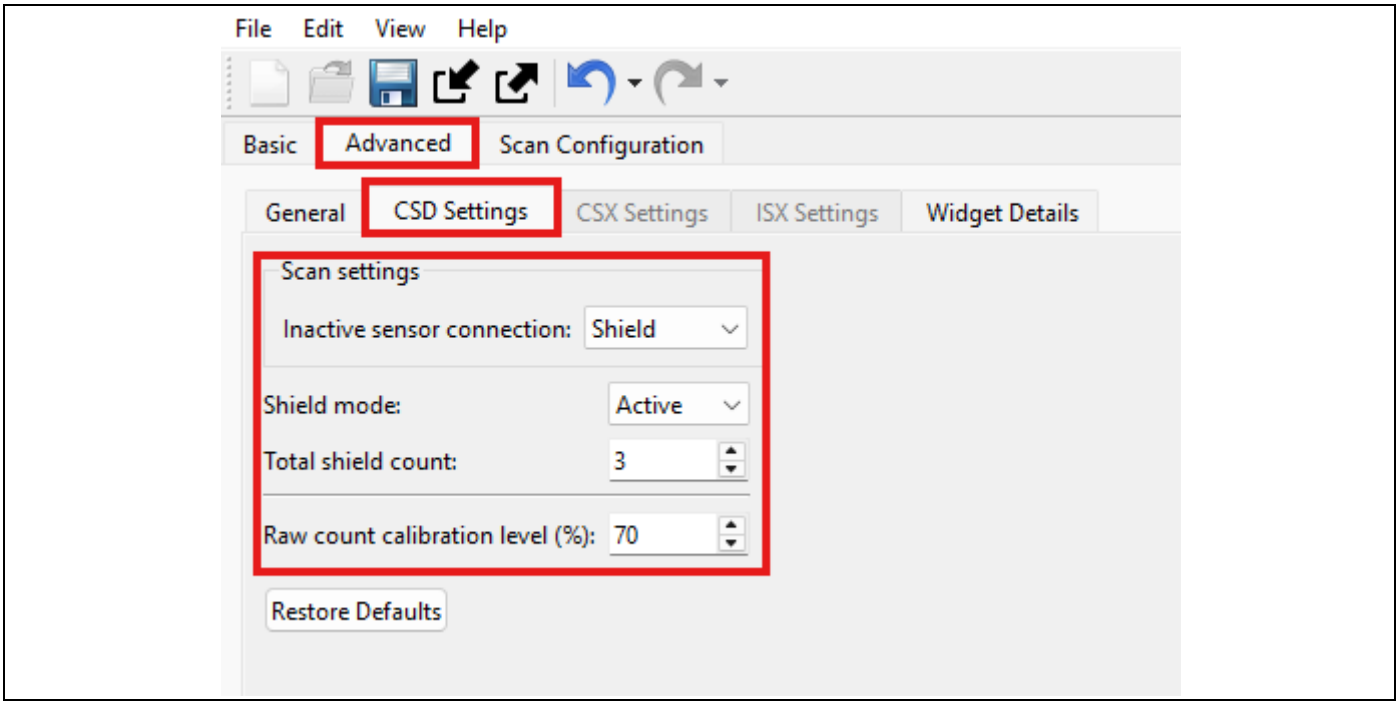

Figure 11 CAPSENSE™ Configurator CSD Settings tab

Parameter	Setting	Comment
Inactive sensor connection	Shield *	Connecting inactive sensors to the shield reduces Cp
Shield mode	Active	The driven shield is a signal that replicates the sensor-switching signal. It helps reduce sensor parasitic capacitance
Total shield count	Equal to the number of shield electrode in the design	Selects the number of shield electrodes used in the design. Most designs work with one dedicated shield electrode but some designs require multiple dedicated shield electrodes to ease the PCB layout routing or to minimize the PCB area used for the shield layer
Raw count calibration level	Default: 70%	If the sensor raw count saturates (equals Max Raw count) on touch, reduce the Raw count calibration level (%). This will prevent raw count saturation
*** If CSX (mutual capacitance) electrodes are used, then inactive sensors should be connected to GND**		

Table 2 CAPSENSE™ CSD Settings initial parameters

CAPSENSE™ performance tuning

4. Open **Advanced** → **Widget Details** to modify each individual sensor widget configuration.

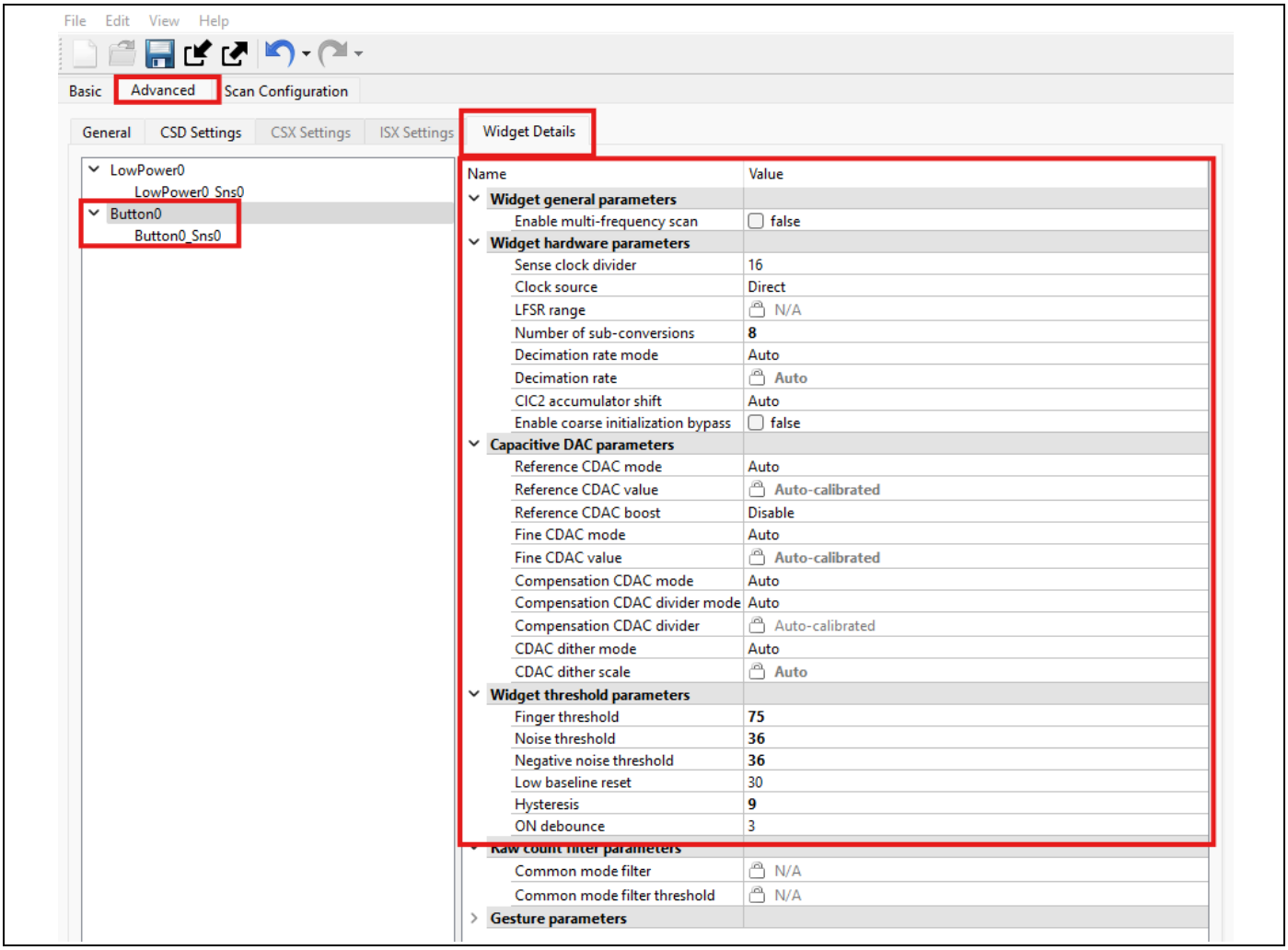

Figure 12 CAPSENSE™ Configurator Widget Details tab

Parameter	Setting	Comment
Sense clock divider	Default: 16	Value is set in Stage 2
Clock source	Default: Direct	Direct clock is a constant frequency sense clock source. When you chose this option, the sensor pin switches with a constant frequency. Spread spectrum clock (SSC) or PRS clock can be used as a clock source to deal with EMI/EMC issues.
Number of sub-conversions	8	A good starting point to ensure a fast scan time and sufficient signal. This value will be adjusted as required in Stage 4
Decimation rate	Default: Auto	Setting this to Auto will calculate the Decimation rate parameter automatically

CAPSENSE™ performance tuning

Finger threshold	Default: 75	
Noise threshold	Default: 10	Baseline is not updated when raw count is above baseline + Noise threshold.
Negative noise threshold	Default: 10	Baseline is not updated when raw count is above baseline + Noise threshold.
Low-baseline reset	Default: 10	If Raw Count is lower than the Negative Noise Threshold for this many samples, baseline is reset to current raw count.
Hysteresis	Default: 10	Prevents sensor status toggling due to system noise.
ON debounce	Default: 3	Number of consecutive scans during which a sensor must be active so that a touch is reported.

Table 3 CAPSENSE™ Widget Settings initial parameters

Note: Widget threshold parameters will be adjusted as required in Stage 5.

CAPSENSE™ performance tuning

5. Open Advanced → Scan Configuration to setup the GPIO connections for the sensor, CMOD, and shield/s.
    - Configure pins for the electrodes using the drop-down menu
    - Configure the scan slots using the Auto-assign slots option. It automatically reassigns all slots for the sensors based on a widget and sensor order
    - Select Button0_Sns0 as Ganged under the LowPower0 widget
    - Check the notice list for warnings or errors

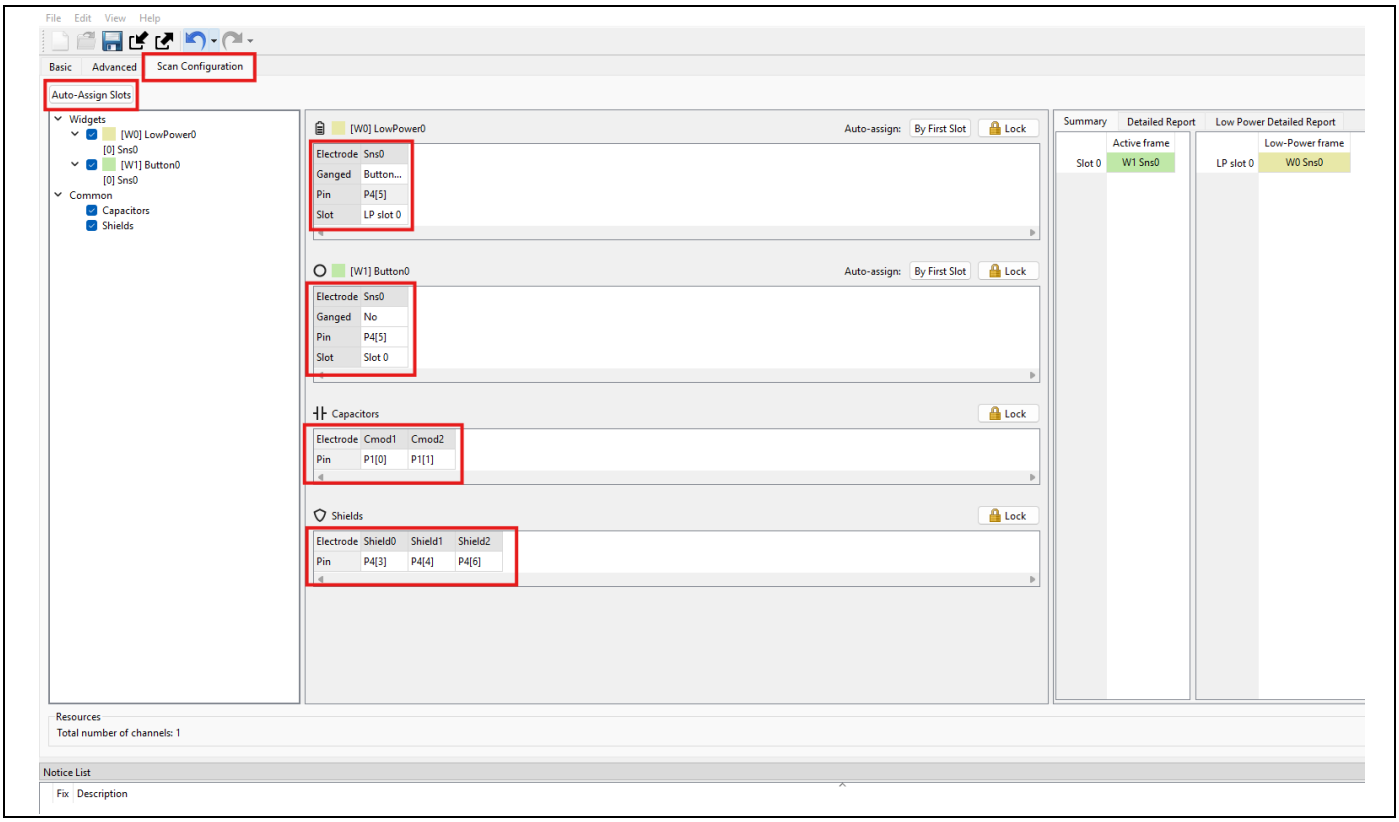

Figure 13 CAPSENSE™ Configurator Scan Configuration tab

6. Click Save to apply the settings

CAPSENSE™ performance tuning

7. Program the device by pressing the "**MSCLP_Low_Power_CSD_Button Program**" in the Eclipse IDE quick panel under **Launches.**

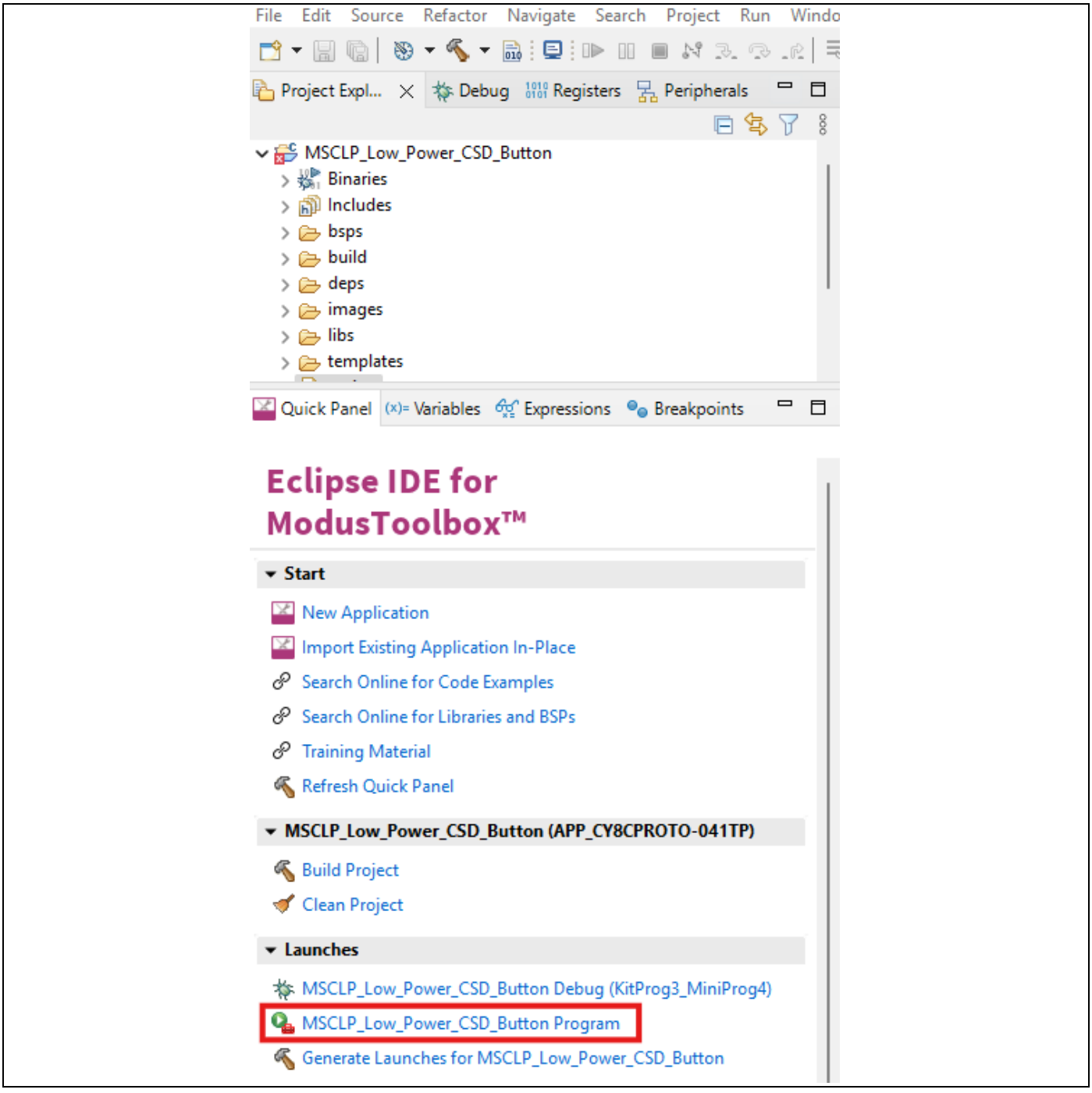

Figure 14 Programming the device with the application from Eclipse IDE

3.3.3 Stage 2 - Set the sense clock frequency

Once all of these items are configured with initial values, program the device and begin measuring the charge and discharge cycle on the electrode so that the sense clock can be configured properly. The sense clock is derived from the modulator clock using a clock-divider and is used to scan the sensor by driving the CAPSENSE™ switched capacitor circuits. Both the clock source and clock divider are configurable. The sense clock divider should be configured such that the pulse width of the sense clock is long enough to allow the sensor capacitance to charge and discharge completely. This is verified by observing the charging and discharging waveforms of the sensor using an oscilloscope and an active probe. The sensors should be probed on the electrode as far away from the device as possible. If a shield is used, then the shield can be used to check that the proper charge and discharge cycle is occurring. If the shield is being probed, it should be probed as far away from the device as possible.

The proper charge and discharge of the sensor occurs when the pulse width at each charging phase is 5 Tau and the voltage is reaching 99.3% of the required voltage at the end of each phase.

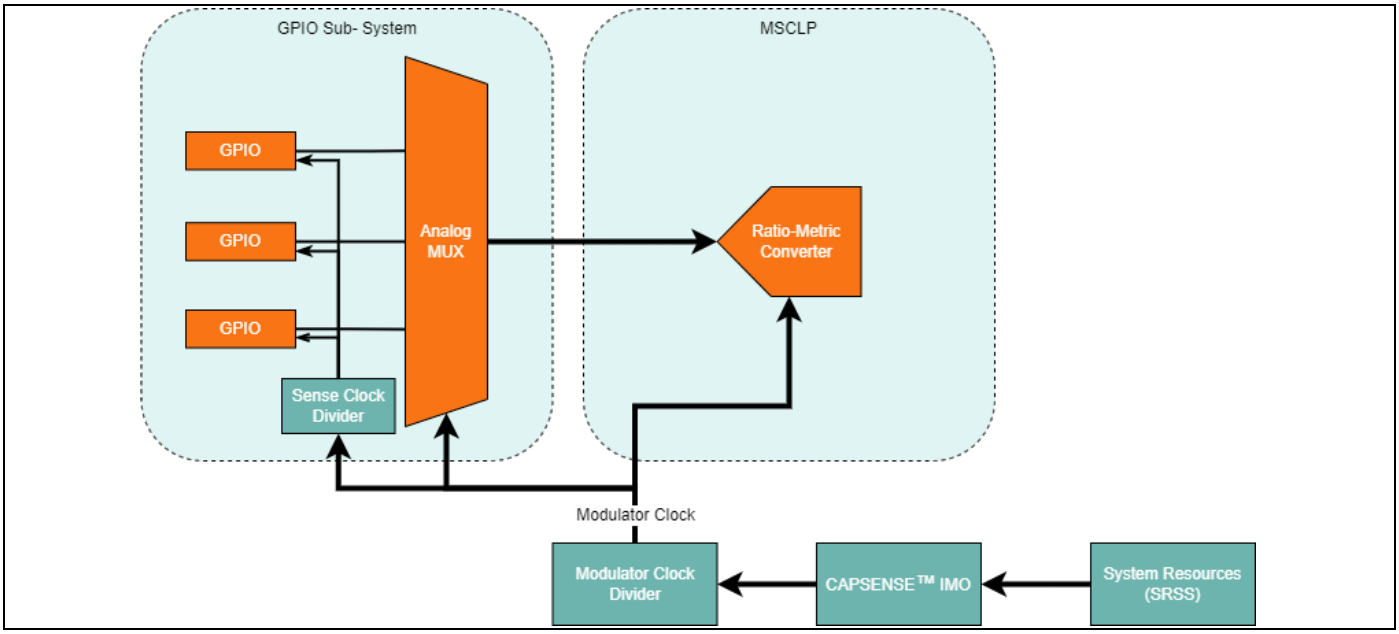

Figure 15 MSCLP Clock Tree

CAPSENSE™ performance tuning

The sense clock frequency must be configured to allow for proper charging and discharging of the sensor. A slower sense clock frequency will allow for sensors with larger parasitic capacitances to charge and discharge properly. The charge/discharge cycle must be observed using an oscilloscope using active probes.

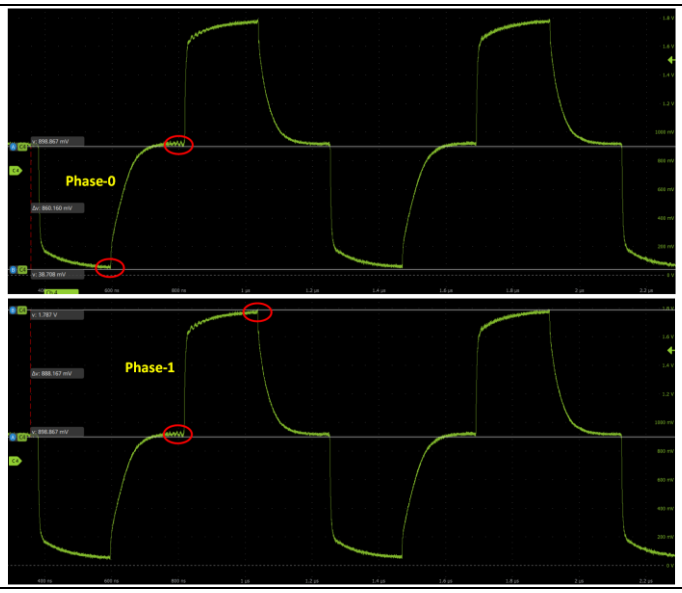

Figure 16 Proper charge cycles observed with an oscilloscope

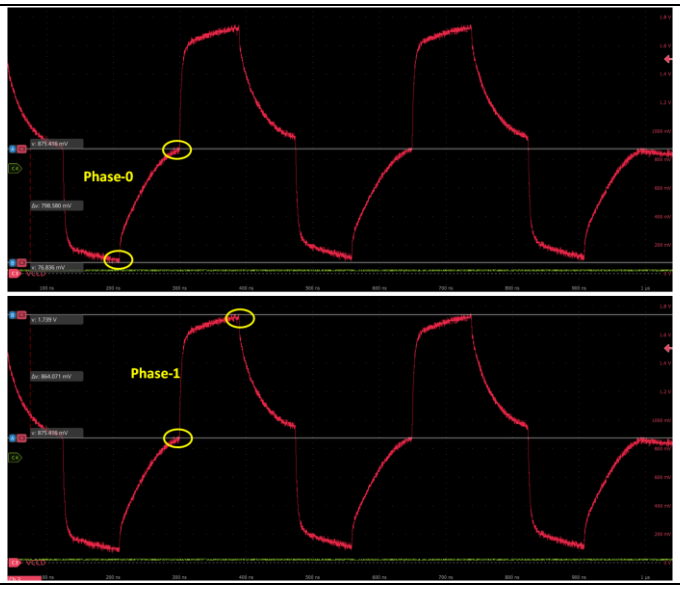

Figure 17 Improper charge cycles observed with an oscilloscope

CAPSENSE™ performance tuning

Without the proper charge/discharge cycle, the signal read by the CAPSENSE™ MSCLP will be noisy. Follow these steps for tuning the sense clock divider.

1. Program the board and launch the CAPSENSE™ Tuner

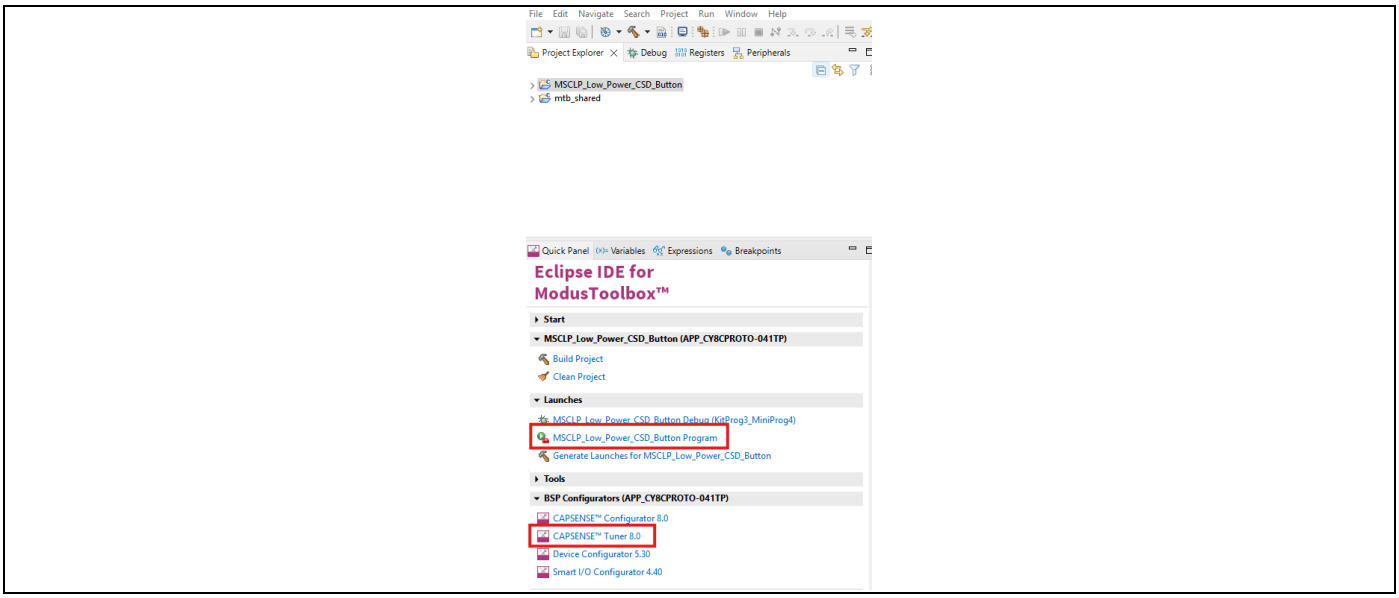

Figure 18 Launching the CAPSENSE™ Tuner from Eclipse IDE quick panel

2. Observe the charging waveform of the sensor as described earlier
3. If the charging is incomplete, increase the sense clock divider. This can be done in CAPSENSE™ Tuner by selecting the sensor and editing the sense clock divider parameter in the Widget/Sensor Parameters panel
    - The sense clock divider should be divisible by 4. This ensures that all four scan phases have equal durations
    - After editing the value click the Apply to Device button and observe the waveform again. Repeat this until complete settling is observed
    - Using a passive probe will add an additional parasitic capacitance of around 15 pF; therefore, it should be considered during tuning

Note: You may need to configure the tuner before starting the connection, this can be done by navigating to the **Tools → Tuner Communication Setup** in the top ribbon. Select your KitProg3 and choose I2C. Ensure that the Port configuration also match.

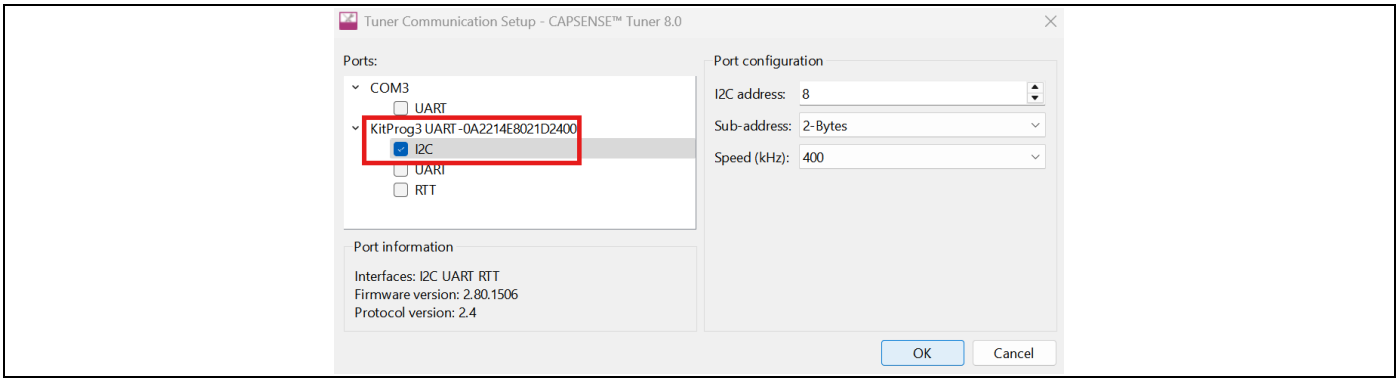

Figure 19 CAPSENSE™ Tuner communication setup

CAPSENSE™ performance tuning

4. Click the Apply to Project button so that the configuration is saved to your project

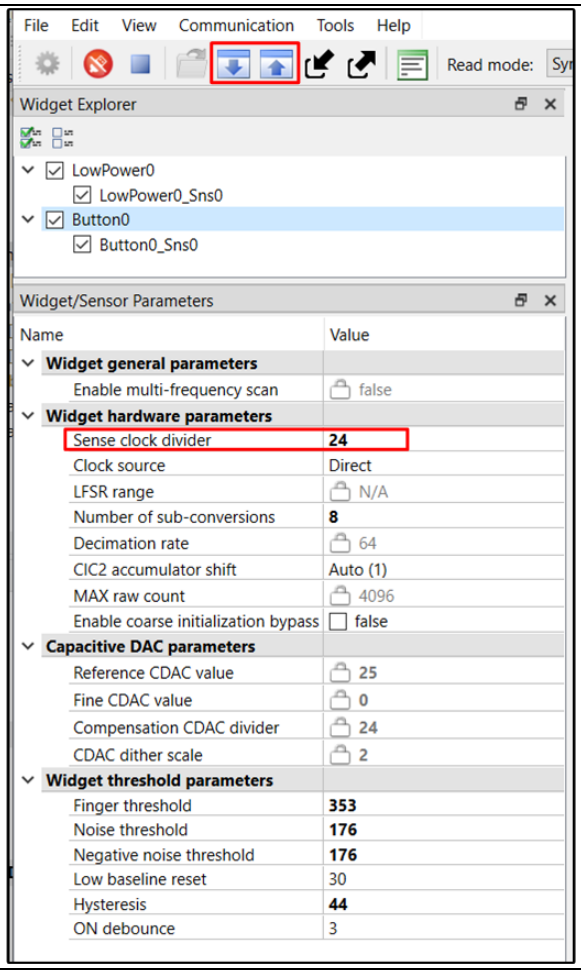

Figure 20 CAPSENSE™ Tuner Widget/Sensor Parameter sense clock divider

5. Repeat this process for all the sensors and the shield. Each sensor might require a different sense clock divider value to charge/discharge completely. But all the sensors which are in the same widget need to have the same sense clock source, sense clock divider and number of sub-conversions. Therefore, take the largest sense clock divider in a given widget and apply it to all the other sensors in the widget

CAPSENSE™ performance tuning

3.3.4 Stage 3 - Measure sensor capacitance to set CDAC tuning mode

Generally, the CDAC tuning mode is recommended to be set to **Auto**, however the appropriate tuning mode to use has some dependency on the sensor parasitic capacitance (Cp).

In order to avoid signal variation across devices in production, PSOC™ 4100T Plus devices have CDAC trim codes in Supervisory Flash (SFlash; read-only). This code is used to scale the Reference CDAC and Fine CDAC parameters, which compensates for variations in the CDAC and brings down the overall signal variation across units.

This trimming is only applicable when tuning CSD widgets in active and low power modes when the sensor parasitic capacitance is less than 4 pF. When using the trimming codes to scale the reference and fine CDAC values, the reference and fine CDAC modes should be set to **Manual**.

The CAPSENSE™ middleware provides built in self-test (BIST) APIs to measure the capacitance of sensors configured in the application. If the BIST is run for a sensor and the result shows that a sensors parasitic capacitance is below 4 pF, CDAC scaling should be enabled and the CDAC values should be manually configured.

1. Measure the sensor Cp using the CAPSENSE™ middleware, it provides Built-In Self-Test (BIST) APIs to measure the capacitance of sensors configured in the application

    - Open CAPSENSE™ Configurator from Quick Panel and enable the library

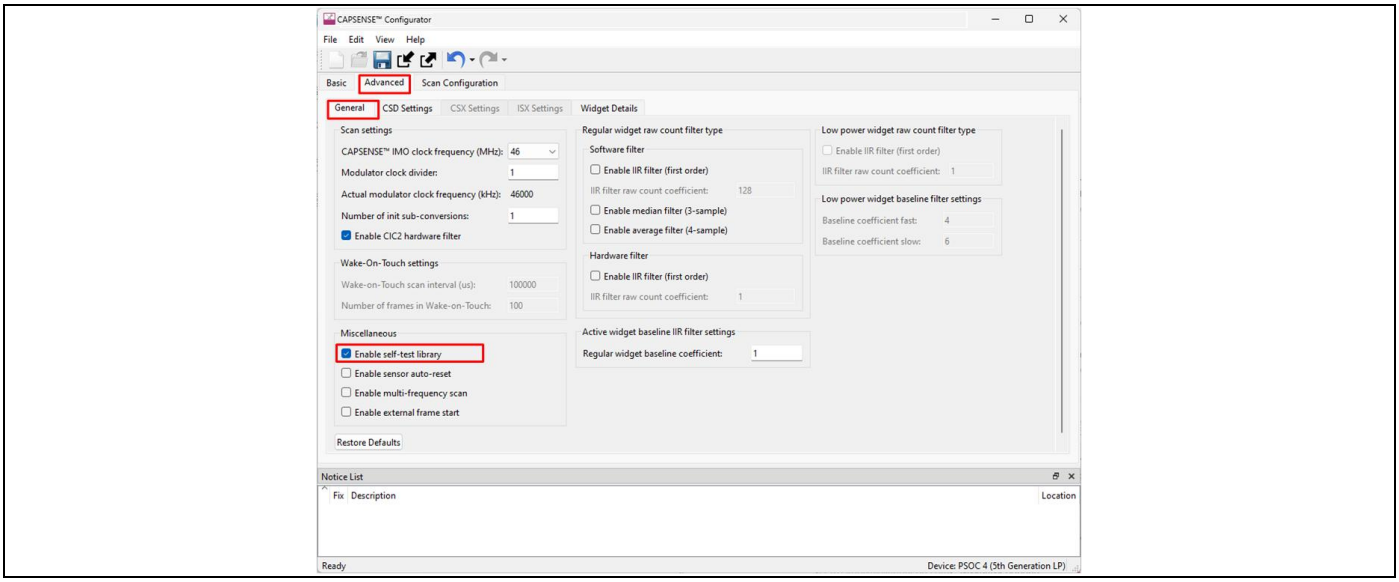

Figure 21 Enabling BIST in the CAPSENSE™ Configurator

- The middleware API used to measure the capacitance of the CSD widget shown below should be placed before the main.c for loop, but after the CAPSENSE™ initialization

```
/* Measure sensor capacitance of the CSD widgets */
Cy_CapSense_RunSelfTest(CY_CAPSENSE_BIST_SNS_CAP_MASK, &cy_capsense_context);
```

- Program the device

CAPSENSE™ performance tuning

- Open the tuner and check the sensor capacitance (Cp) values. If Cp value is above 4 pF, set all CDAC parameters to **Auto** and proceed to **Stage 4 tuning**

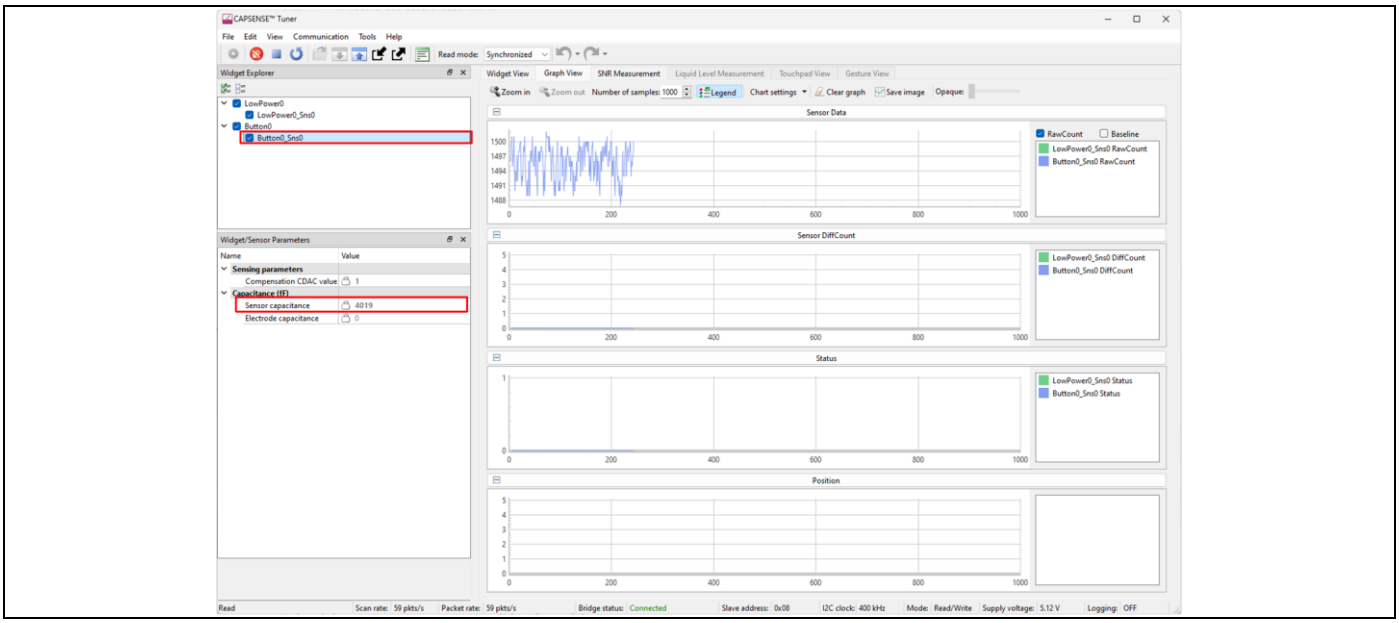

Figure 22 Observing measured sensor capacitance in the CAPSENSE™ Tuner

Note: Remove the Self-Test API once the sensor capacitance is measured

2. Enabling CDAC scaling and setting manual CDAC values, if the sensor Cp is below 4 pF

- Open CAPSENSE™ Configurator and start by setting all CDAC parameters to Auto

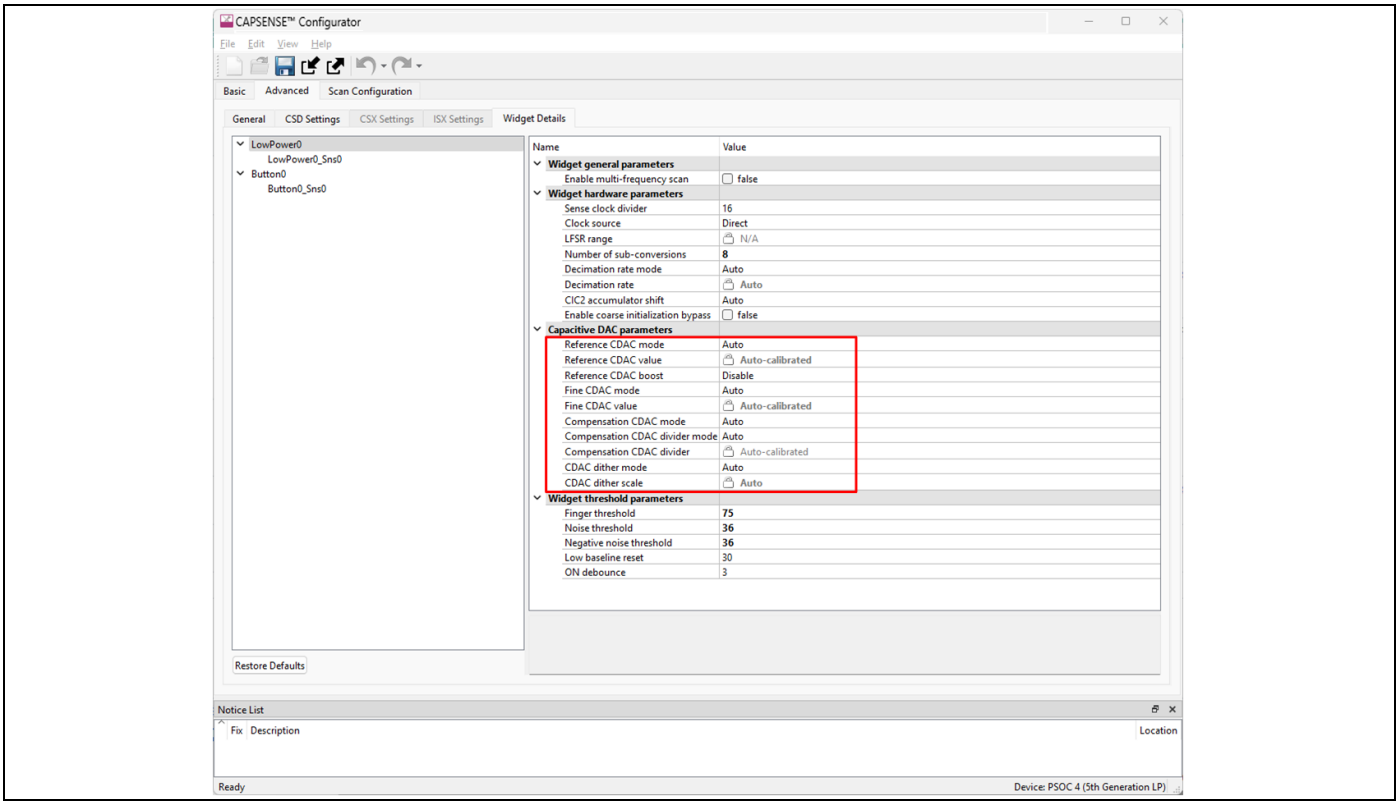

Figure 23 Setting CDAC parameters to Auto in the CAPSENSE™ Configurator

CAPSENSE™ performance tuning

- Program the device. Run the CAPSENSE™ Tuner and click apply to project button. This applies the auto calculated CDAC values to the project configuration

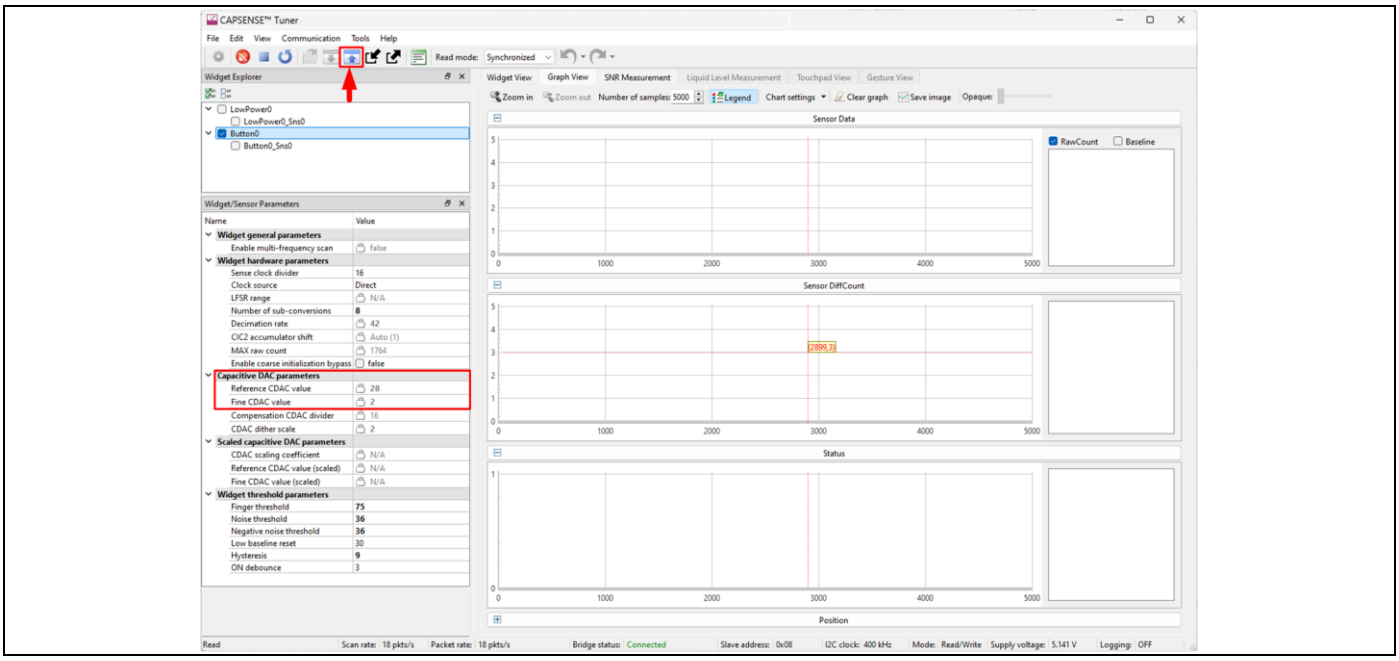

Figure 24 Applying CDAC parameters from the CAPSENSE™ Tuner

- Close and reopen the CAPSENSE™ Configurator, and enable CDAC scaling

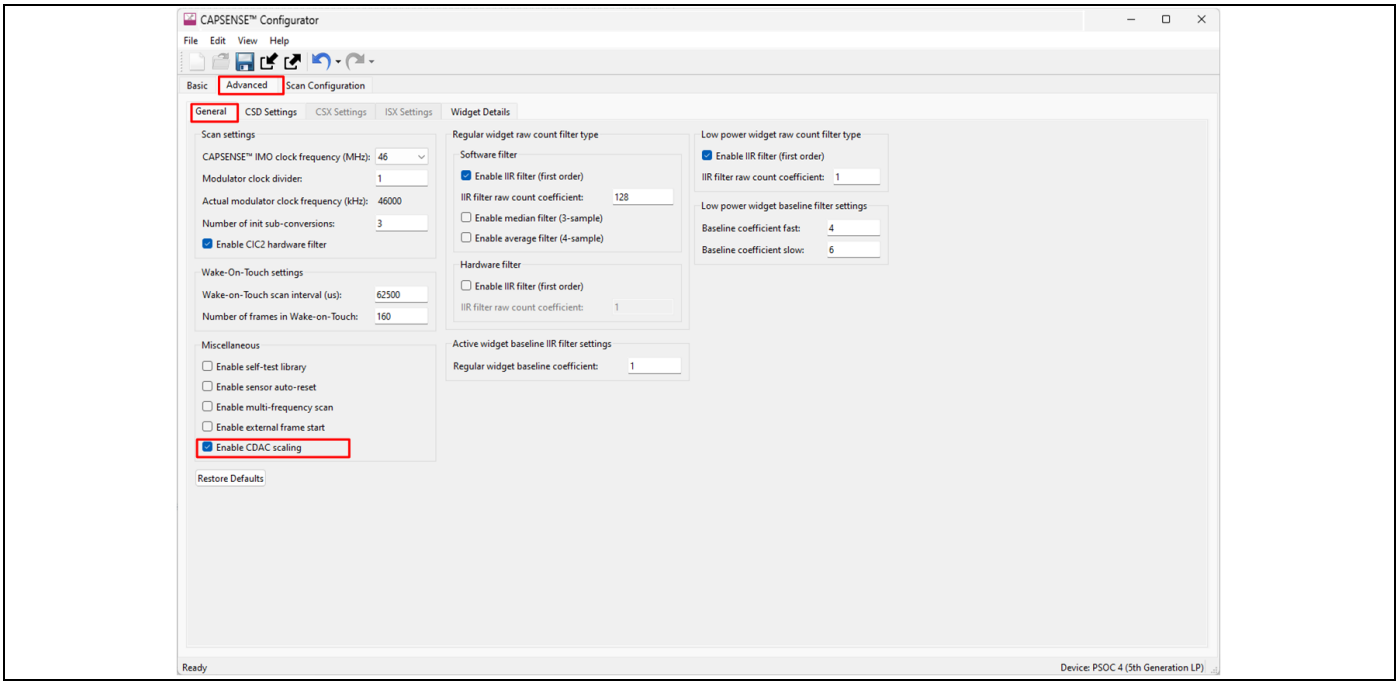

Figure 25 Enabling CDAC scaling from the CAPSENSE™ Configurator

- Set Manual tuning mode only for Reference CDAC mode and Fine CDAC mode parameters. You can see the CDAC values observed in Figure 24 when CDAC was set to **Auto**

CAPSENSE™ performance tuning

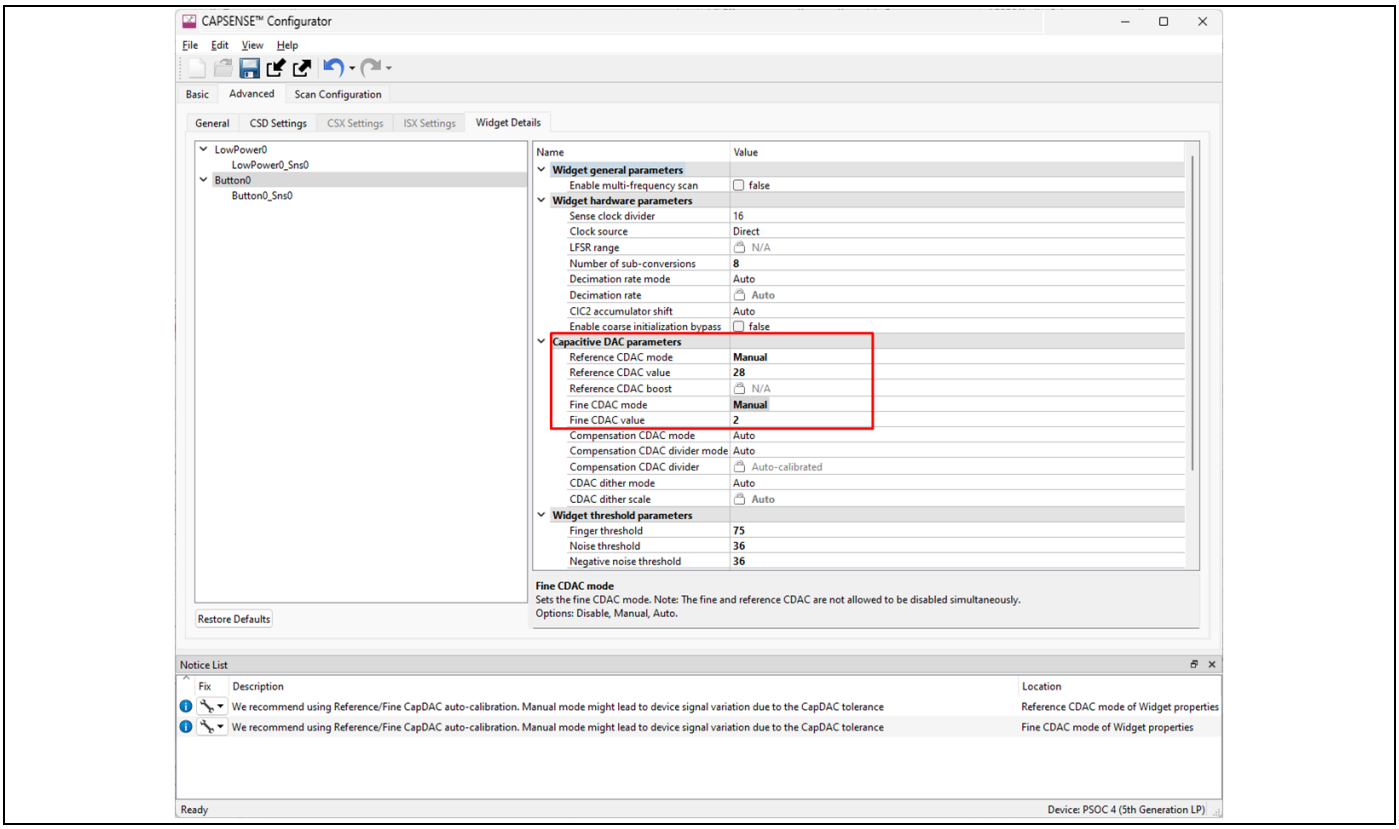

Figure 26 Setting CDAC parameters manually in the CAPSENSE™ Configurator

CAPSENSE™ performance tuning

- Flash the device with updated CAPSENSE™ configuration
- Open CAPSENSE™ Tuner. Now you can see the scaled parameters after applying CDAC scaling

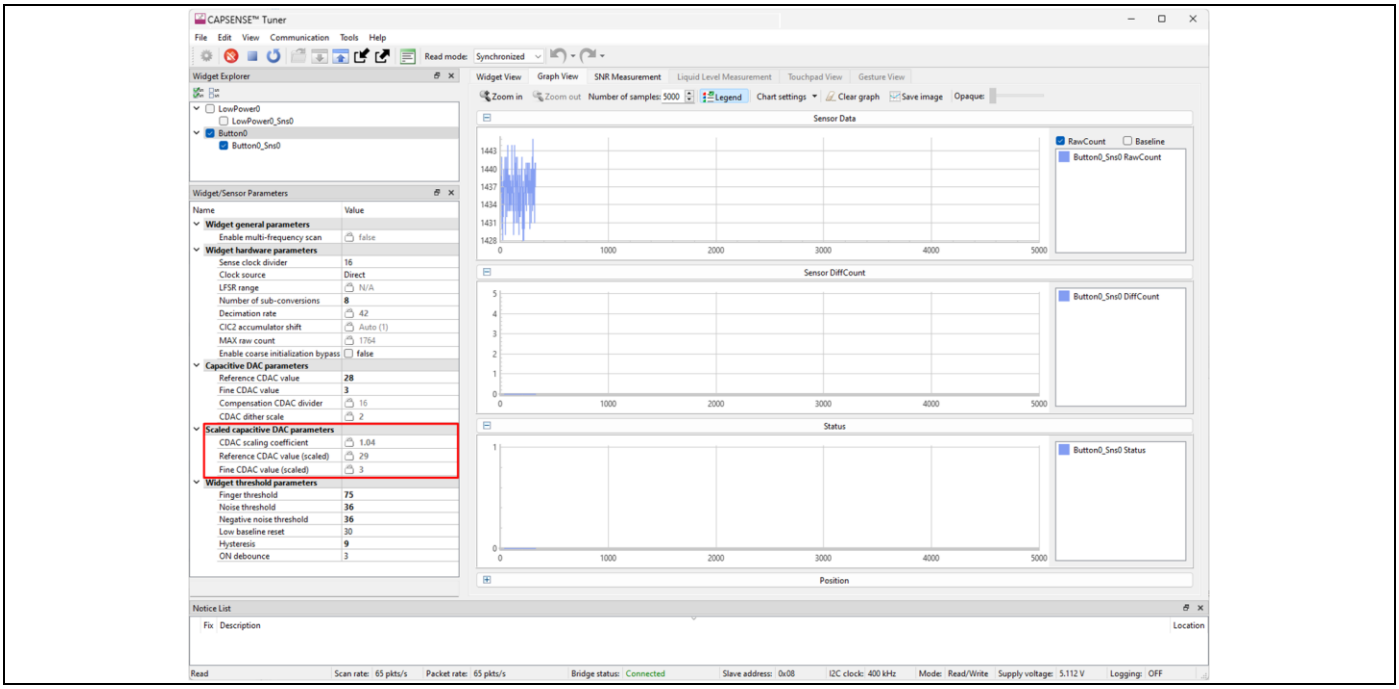

Figure 27 Checking that CDAC parameters were set in the CAPSENSE™ Tuner

- Use this as a base CDAC configuration and proceed with tuning

CAPSENSE™ performance tuning

3.3.5 Stage 4 - Fine tune for the required SNR, power, and refresh rate

Once the sense clock frequency is set correctly and CDAC tuning is complete, the sensor can be tuned for optimal sensitivity, timing, and power consumption requirements. Using the CAPSENSE™ Tuner, the SNR can be measured. The number of sensor sub-conversions can be increased until the SNR requirements are met. Generally, a 5:1 SNR is the minimum required.

If the system is noisy (> 40% of signal), enable the filters. The PSOC™ 4100T Plus and other devices with the MSCLP block have a raw count filters show in the table below.

Filter	Description
Median	Eliminates noise spikes from motors and switching power supplies
Average	Eliminates periodic noise such as power supply or AC noise
First Order IIR	Software IIR filter which eliminates high frequency noise. A lower coefficient results in lower noise, but increases response time. The coefficient range is 1 to 255.
Hardware IIR	Eliminates high frequency noise. The low coefficient means less filtering, but the response time is higher

Table 4 CAPSENSE™ Filters

The MSCLP Low Power CSD Button example project enables a state machine for application power modes.

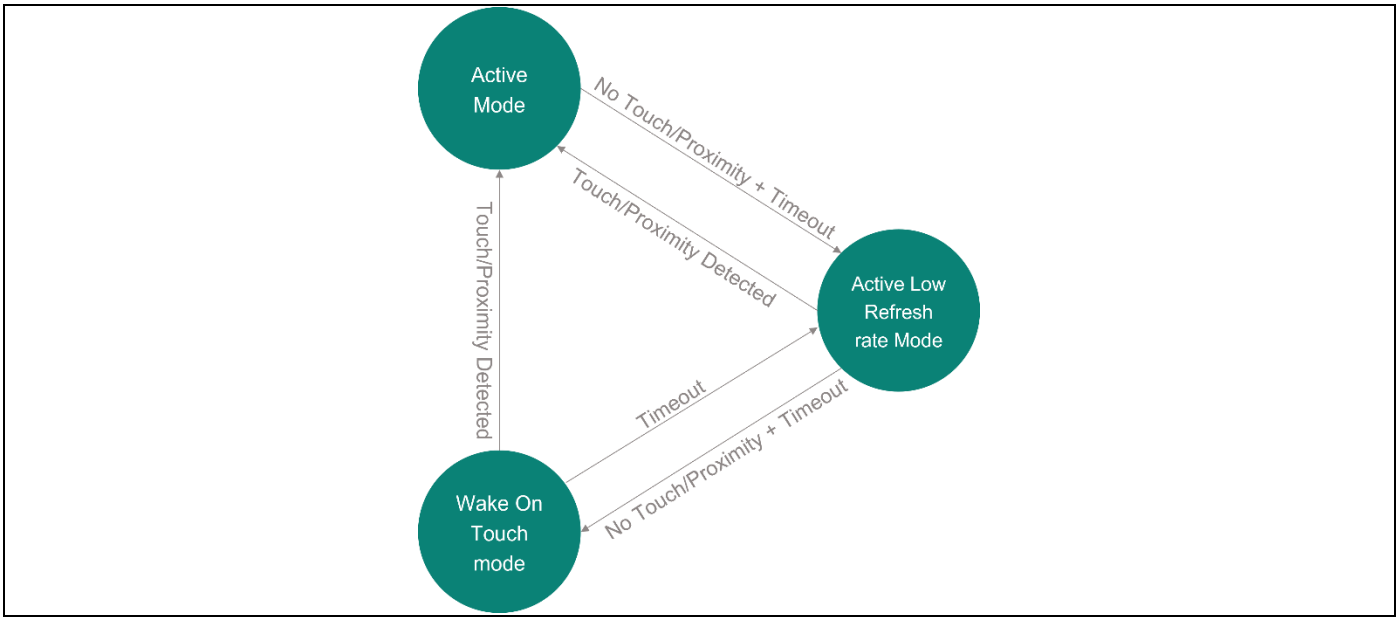

Figure 28 Example Code Application Power State Machine

Along with filtering, the refresh rate for each application power mode can be adjusted to increase or decrease response time and power consumption.

CAPSENSE™ performance tuning

1. Use the CAPSENSE™ Tuner to measure the SNR

    - Navigate to **SNR Measurement** and press **Acquire Noise** while ensuring nothing comes in range of the sensor

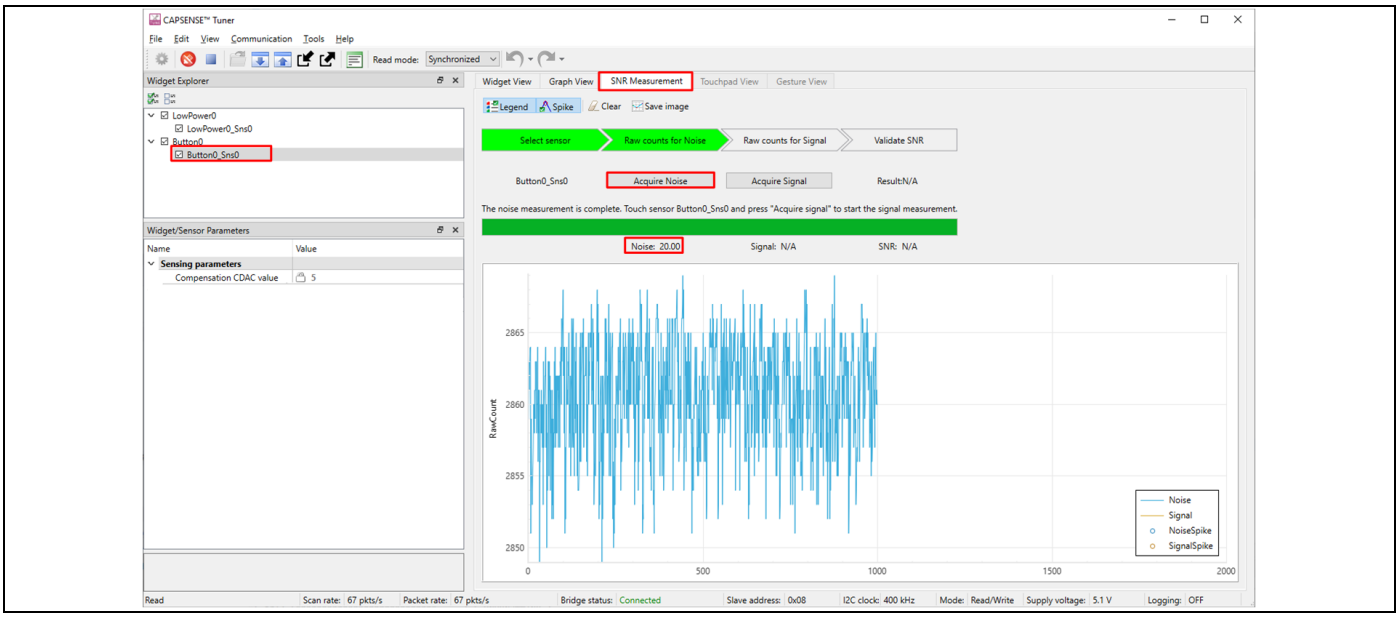

Figure 29 Acquiring Noise in the CAPSENSE™ Tuner

- Once the noise is captured, generate an expected typical touch on the sensor then press **Acquire Signal** while generating the touch the entire time it is capturing the signal

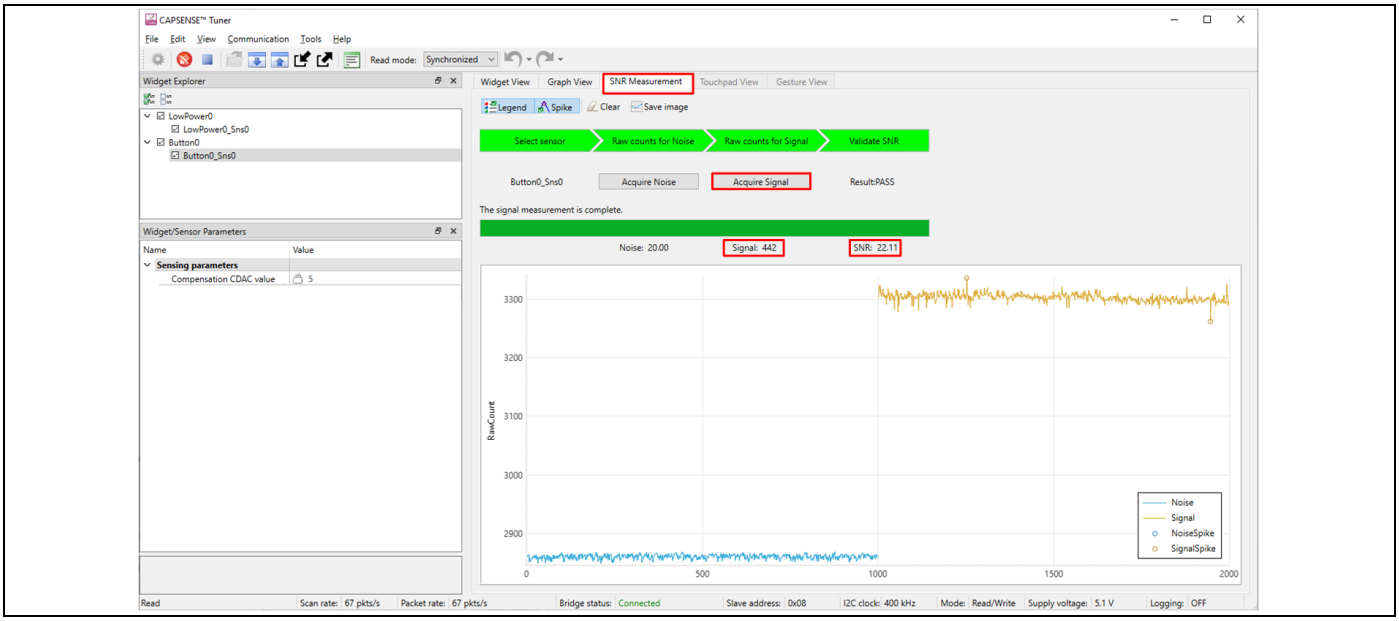

Figure 30 Acquiring Signal in the CAPSENSE™ Tuner

Note: When tuning a low-power sensor, the raw counts are only reported to the tuner while in active or ALR application power modes. This means that while tuning the low-power sensors, data won't appear on the graph until the device enters those modes. Also, when acquiring the signal, it is best to generate the touch, wait for a cycle of active or ALR mode, then to press the **Acquire Signal** button.

CAPSENSE™ performance tuning

2. If the SNR is less than 5:1, increase the number of sub-conversions. Edit the number of sub-conversions (Nsub) directly in the **Widget/Sensor** parameters tab of the CAPSENSE™ Tuner
3. Apply changes to device

Figure 31 Applying sensor settings from the CAPSENSE™ Tuner

- Repeat steps 1 to 3 until the SNR of 5:1 is met and the signal count is greater than 50

4. If the system is noisy (> 40% of signal), enable the filters described in Stage 4 description

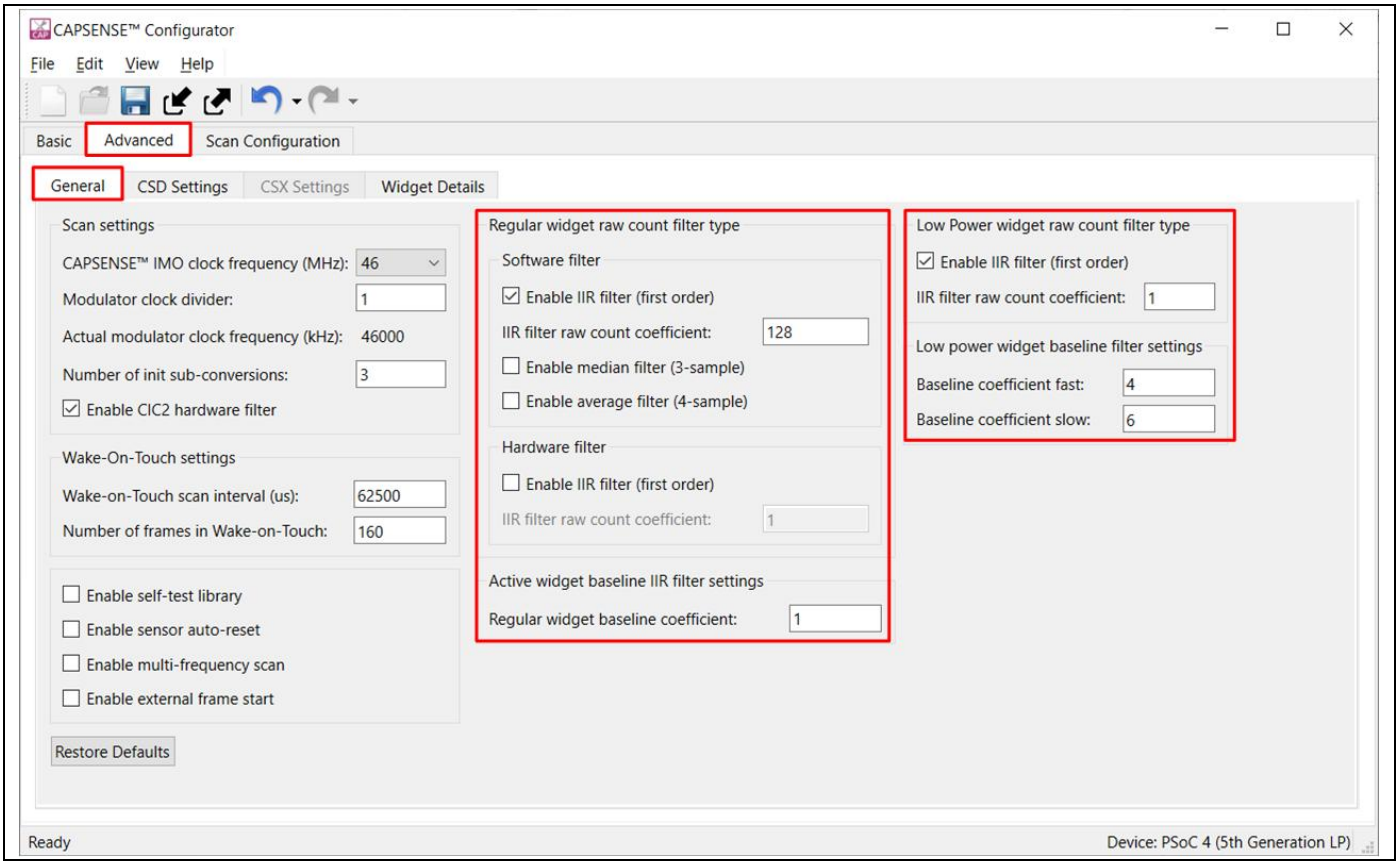

Figure 32 Adjusting filters in the CAPSENSE™ Configurator

5. Click Save and program the device to update the filter settings

CAPSENSE™ performance tuning

3.3.6 Stage 5 - Tune the threshold parameters

Various thresholds relative to the signal need to be set for each sensor. Using the CAPSENSE™ Tuner, the sensor signal during a touch event can be observed. The recommended settings for each threshold can be found in the table below:

Parameter	Recommended Setting
Finger threshold	80% of the touch signal
Noise threshold	40% of the touch threshold
Negative noise threshold	40% of the touch threshold
Low baseline reset	30 (by default)
Hysteresis	10% of the touch signal
ON Debounce	3

Table 5 Recommended Threshold Settings

1. In the CAPSENSE™ Tuner, switch to the **Graph View** tab and select the widget to be tuned
2. Touch the sensor widget and monitor the touch signal in the **Sensor Signal** graph

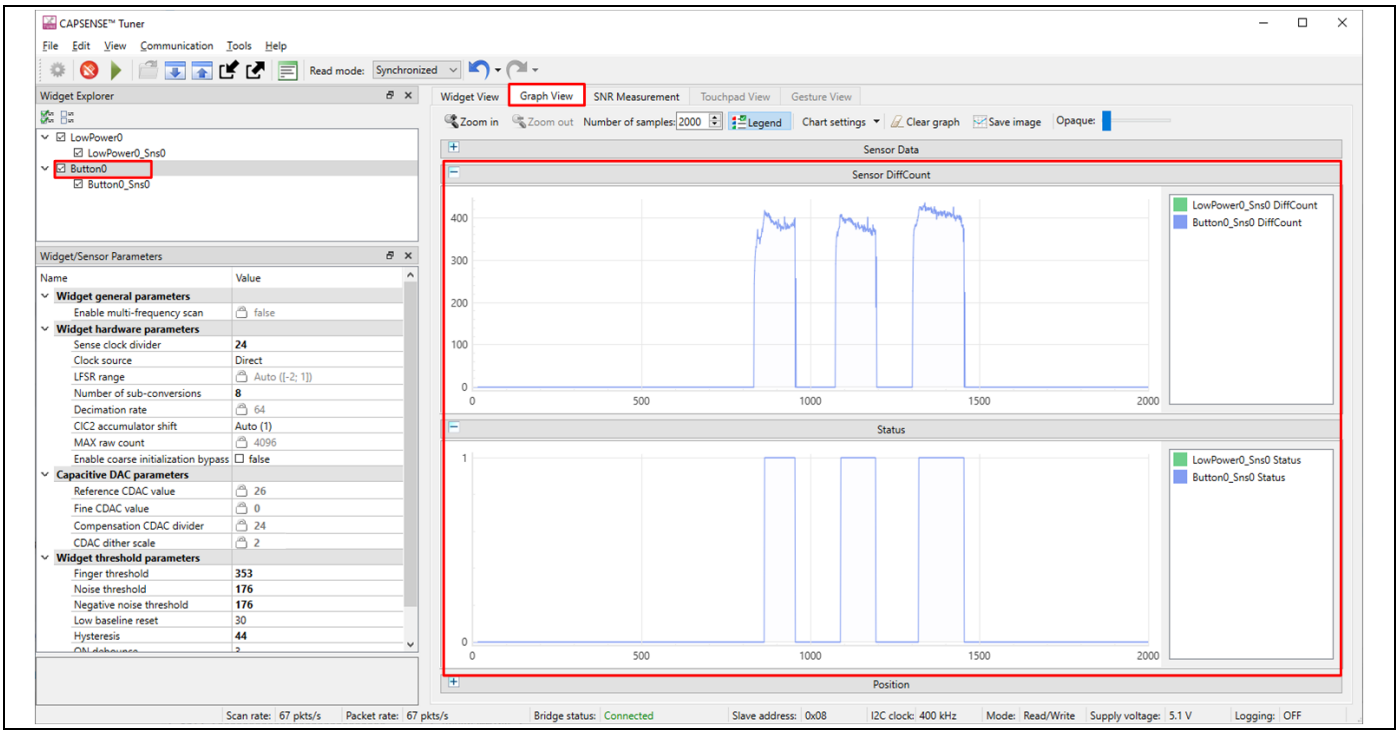

Figure 33 Observing touch signal with the CAPSENSE™ Tuner graph view

CAPSENSE™ performance tuning

3. Using the recommendation in the Stage 5 description, set the thresholds and apply them using the CAPSENSE™ Tuner

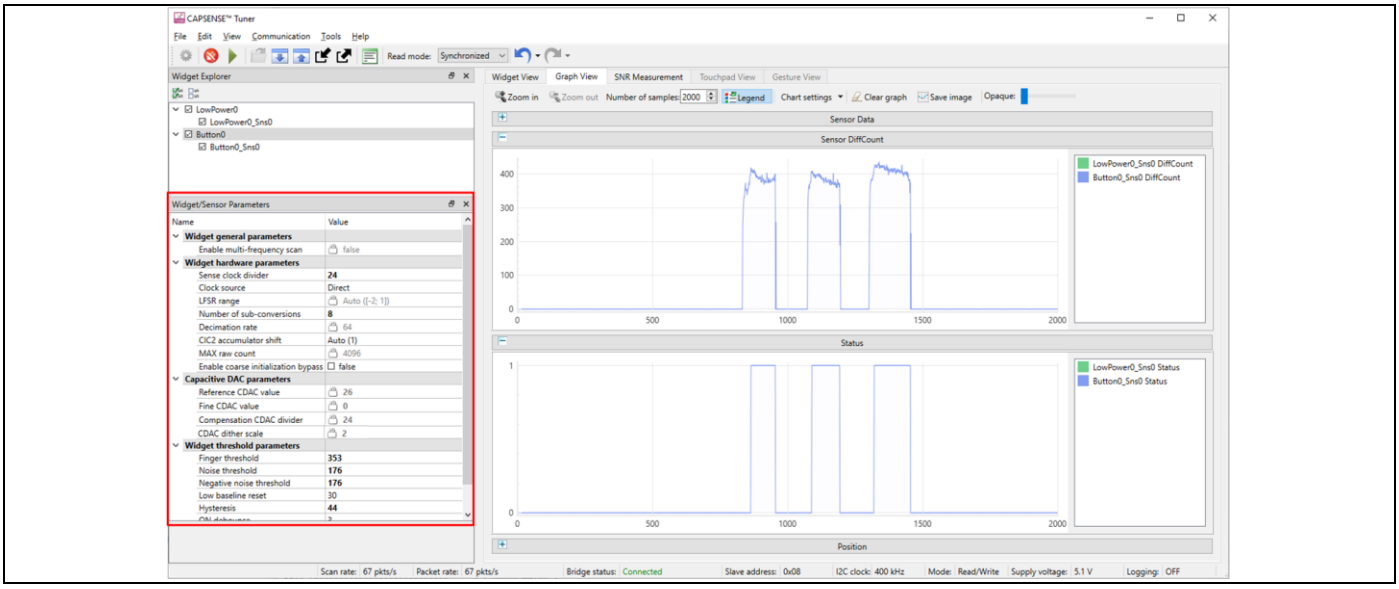

Figure 34 Adjusting thresholds using the CAPSENSE™ Tuner widget/sensor window

4. For the low-power sensor, set the touch threshold to the maximum (65535) to prevent a touch from waking the device from wake-on-touch mode. Then follow steps 1 to 3 for tuning the low-power sensor thresholds

3.4 Output

After applying the configuration, test the performance by touching the button. If your sensor is tuned correctly, you will observe the touch status go from 0 to 1 in the Status panel of the Graph View tab. The status of the button is also indicated by the green LED in the kit; the green LED turns ON when the button is pressed. The red LED will blink at different rates to indicate which application power mode the device is in; 10 Hz for active mode, 2 Hz for ALR mode, and 1 Hz for Wake-On-Touch mode.

3.5 Conclusion

In this exercise we demonstrated how to tune a self-capacitance sensor using the CAPSENSE™ performance tuning method. Using this iterative method, nearly all self-capacitive sensors can be tuned for optimal performance, considering the applications performance, responsiveness, and power consumption.

4 CAPSENSE™ low power tuning

4.1 Objective

The objective of this lab is to demonstrate how to gang multiple sensors to reduce the low power scan duration as well as demonstrate wake-on-touch versus wake on approach methods of low-power HMI strategies.

4.2 Description

When designing an HMI for a low-power application, such as a wearable device, multiple capacitive sensors are often used. The more capacitive sensors that need to be sensed, the longer the scan duration takes, causing the MSCLP block to be active for a longer period of time. In previous versions of the MSC block, a proximity electrode that typically looped around the entire touch area was needed for proximity wakeup, complicating the layout. A major benefit to using a PSOC™ device with the MSCLP block is the ability to gang sensors together for low power sensing. This allows the MSCLP block to only sense a smaller subset of sensors instead of all sensors required for the HMI.

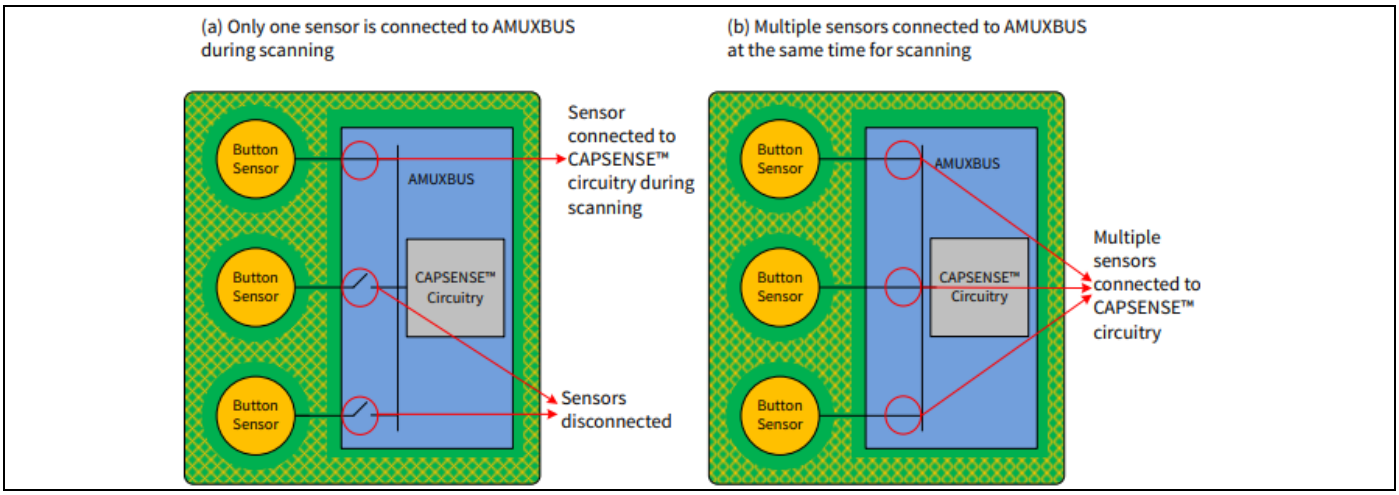

Figure 35 Analog MUX connecting multiple sensors to CAPSENSE™ circuitry

There are two main strategies for low-power sensing:

1. Wake-on-Touch (WoT):
    - If the low-power sensor is tuned to detect touch, a touch event on the low power widget will wake the device such that it begins scanning all sensors independently, which allows it to determine which button was pressed. This method will provide the lowest average power consumption as the refresh rate and scan duration in WoT mode is the lowest and accidental proximity events with the touch area will not wake the device from WoT application power mode.
2. Wake-on-Approach:
    - If the low-power sensor is tuned to detect proximity, a proximity event of any button will wake the device such that it begins scanning all sensors independently. This strategy is useful for applications where a “quick tap” or “double tap” detection is required. Typically, the user must be able to see the touch interface for this type of user experience to be used. This strategy may cause higher average power usage as accidental proximity events near the touch interface will wake the device from the WoT application power mode.

CAPSENSE™ low power tuning

4.3 Adding the CSX Button and accompanying low-power sensor

The same ModusToolbox™ will be used for this lab, but the CSX button will be added.

1. Launch the ModusToolbox™ Device Configurator from the Eclipse IDE quick panel

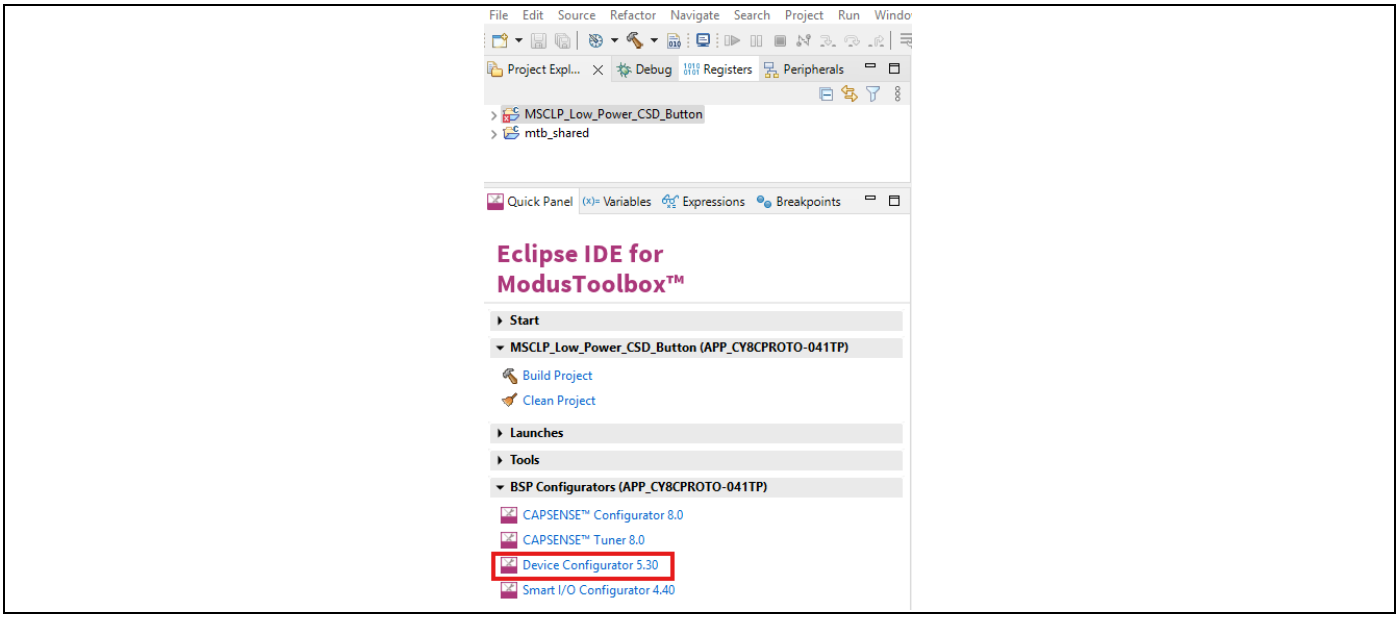

Figure 36 Launching the ModusToolbox™ Device Configurator

2. Navigate to the **Pins** tab and ensure that P4.4 and P4.6 are enabled

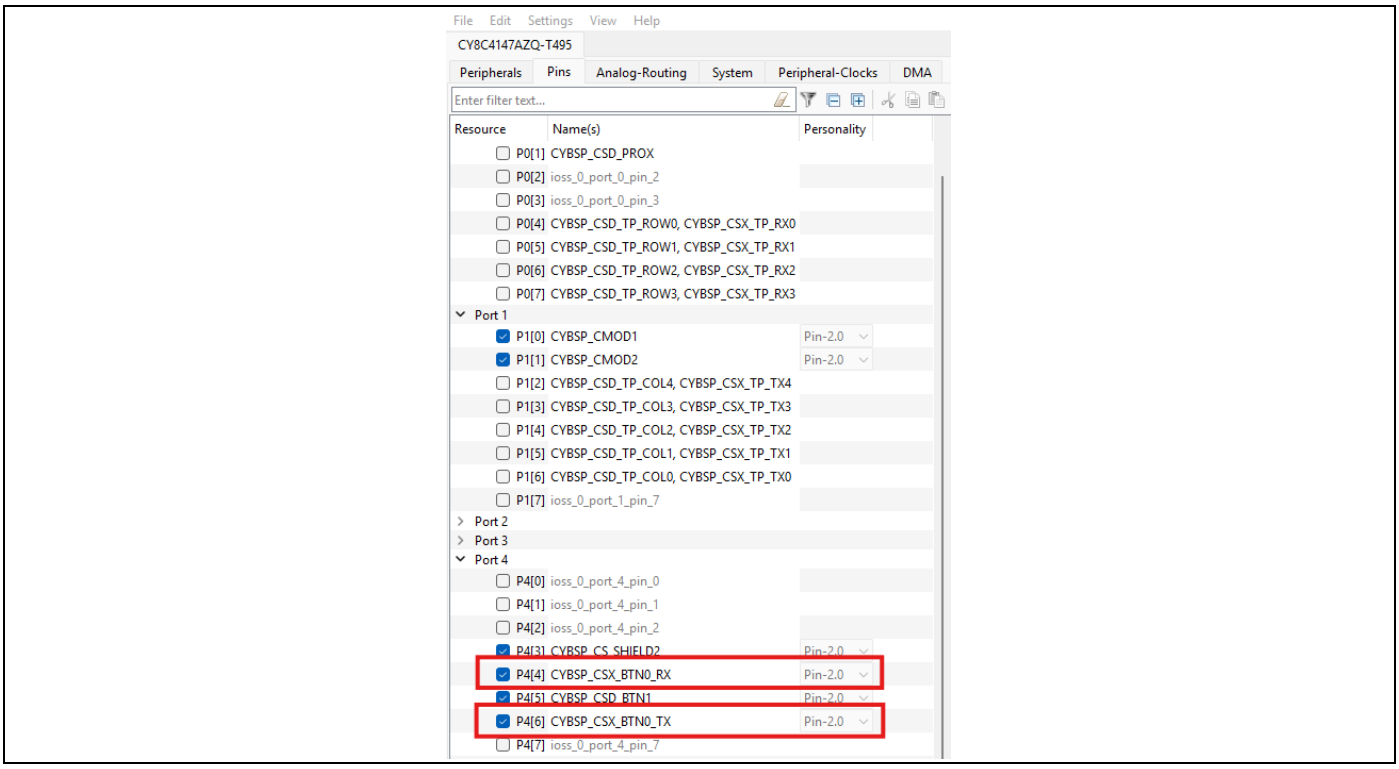

Figure 37 Device Configurator Pins tab

CAPSENSE™ low power tuning

3. Navigate to the **Peripherals** tab in the device configurator, then go to **System → CapSense,** then in the **Parameters** window, press **Launch CAPSENSE™ Configurator**

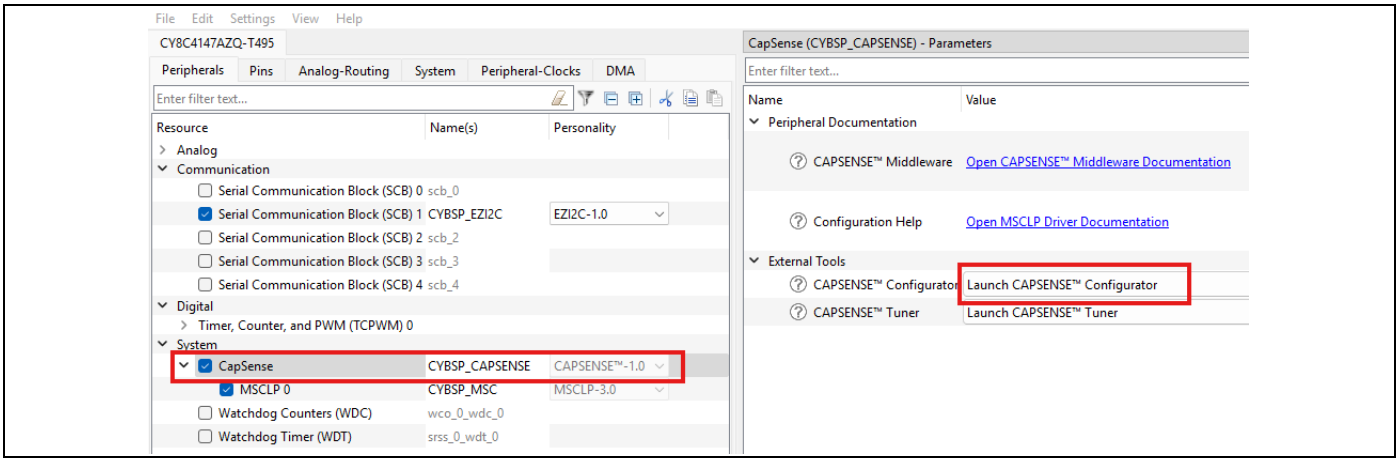

Figure 38 Launch the CAPSENSE™ Configurator from Device Configurator

4. Add the CSX Button and a Low Power from the CAPSENSE™ Configurator **Basic** tab by pressing the **+** button

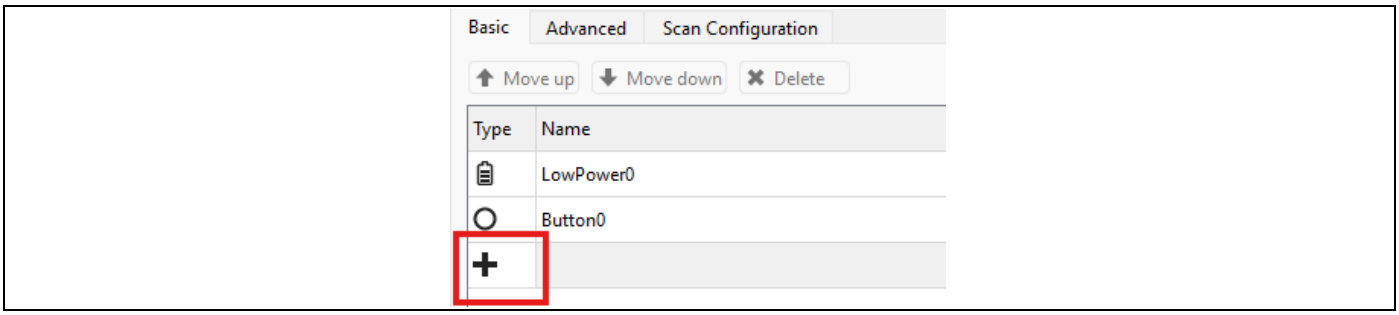

Figure 39 Add button using CAPSENSE™ Configurator

5. Select **CSX RM** as the **Sensing Method** for the new **Button1** and **CSD RM** as the **Sensing Method** for the new **LowPower1** sensor.

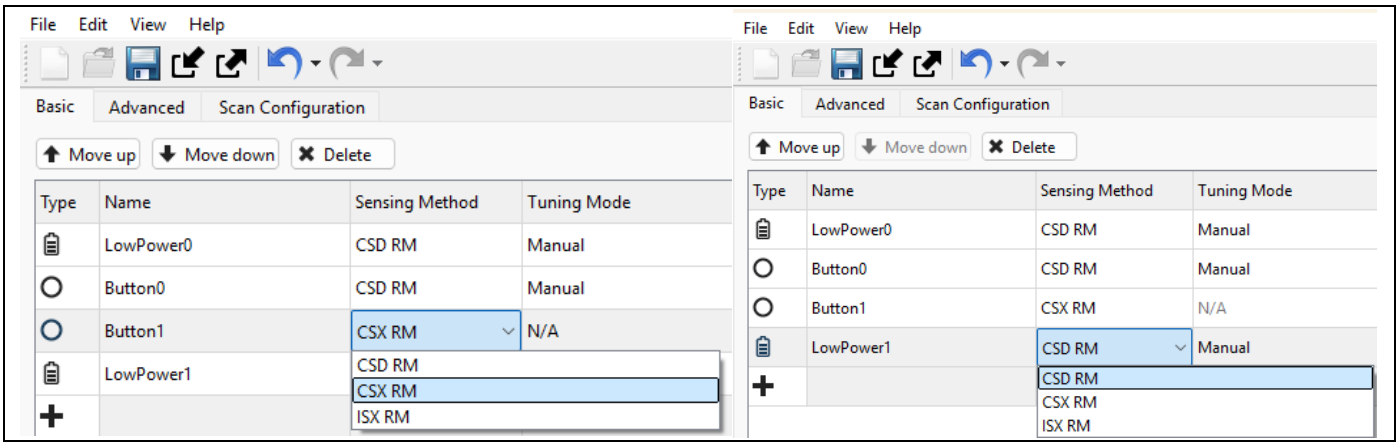

Figure 40 Selecting the sensing method

CAPSENSE™ low power tuning

6. Navigate to the **Advanced** tab and reduce the shield count from 3 to 1. By default, when creating the CSD button example project, the CSX pins are set as shield pins.
7. Navigate to the **Scan Configuration** tab and assign the RX and TX pins for the new **Button1.** Gang the RX and TX pins of **Button1** for the **LowPower1** sensor. Then setup the **shield0** to utilize **P4.3**

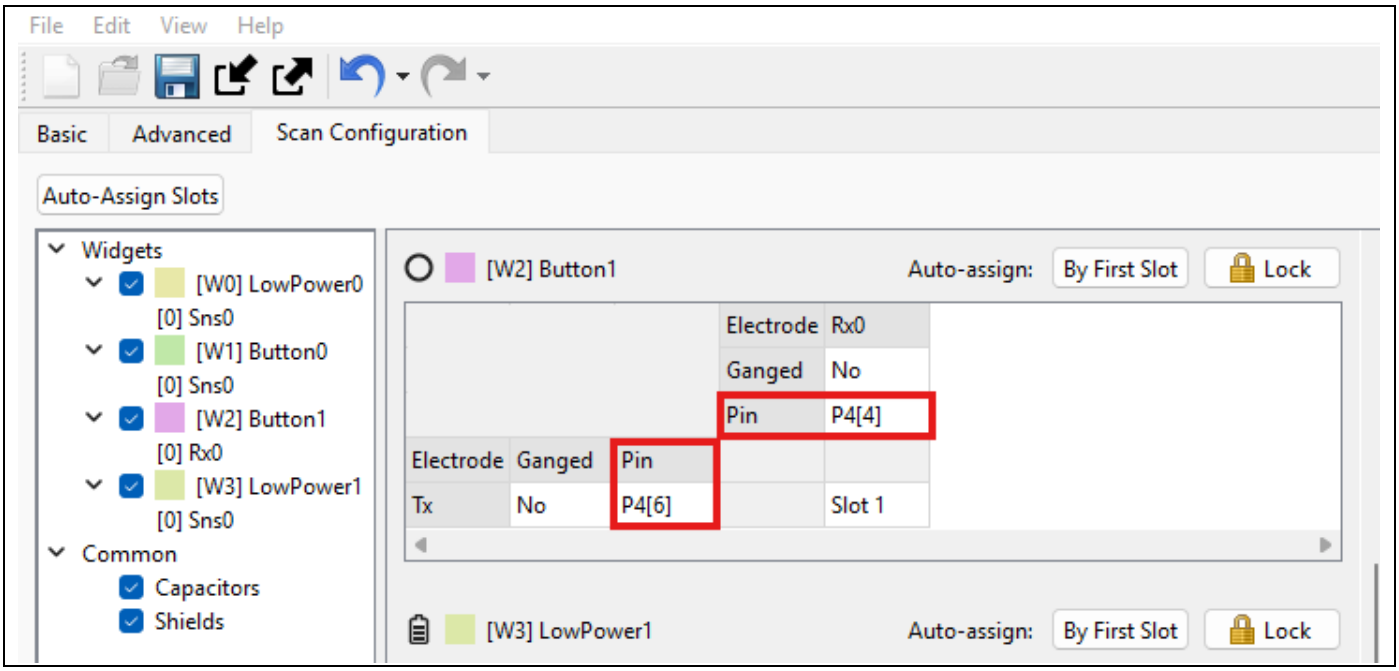

Figure 41 Selecting the RX and TX pins for the new Button1 sensor

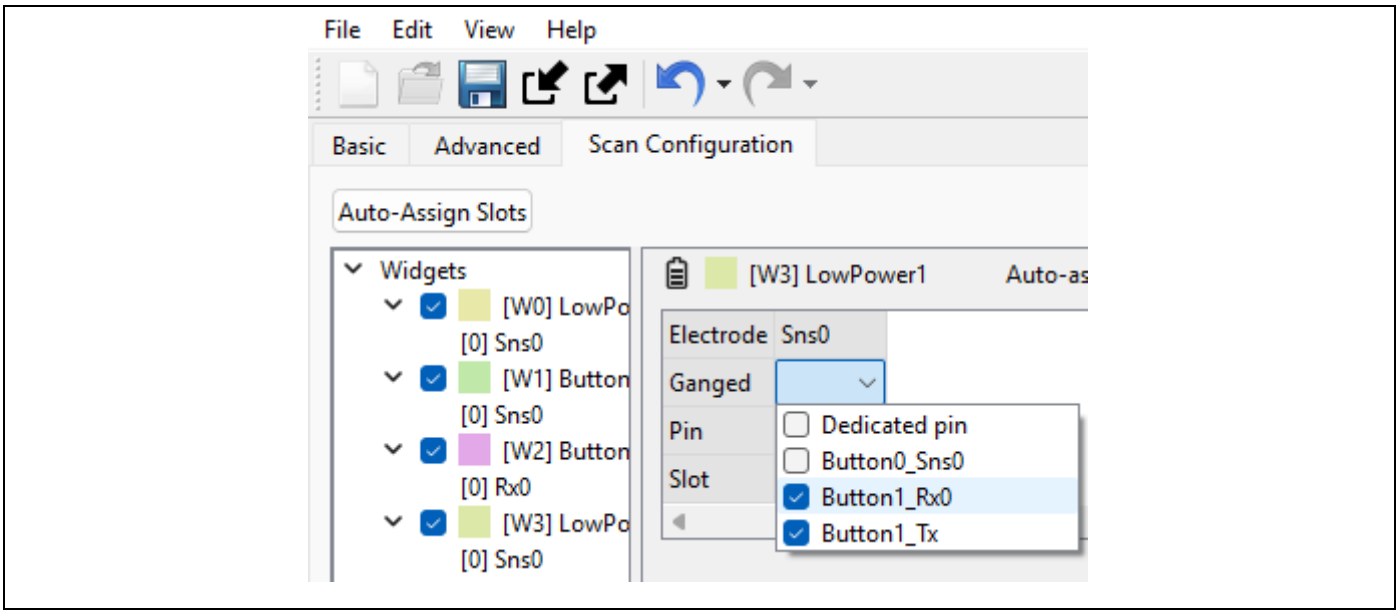

Figure 42 Ganging Button1 RX and TX pins for LowPower1 sensor

CAPSENSE™ low power tuning

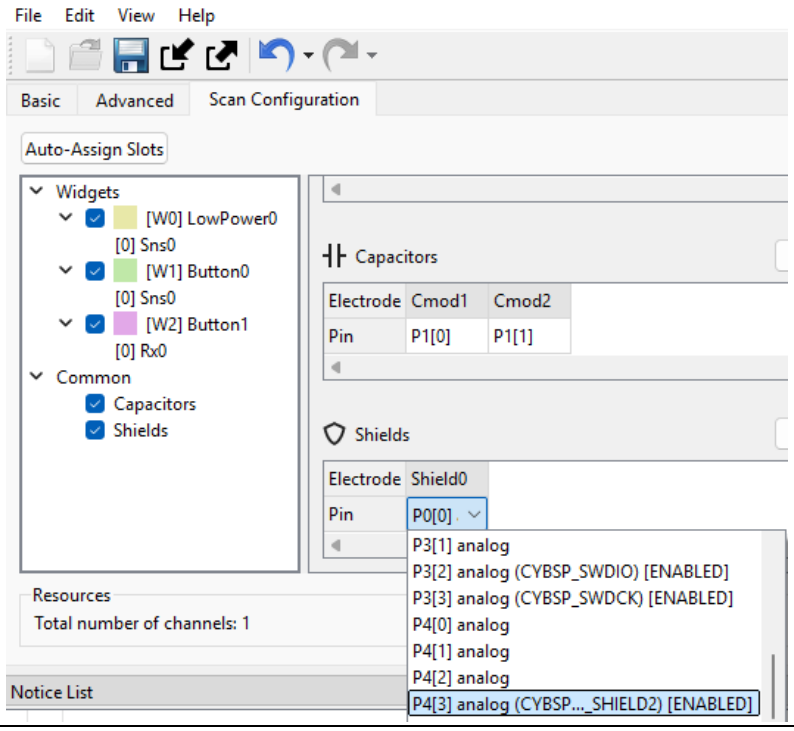

Figure 43 Selecting the shield pin

8. Update the main.c “led_control()” function to enable activating the green LED when touch on the new button is detected

```
if (CAPSENSE_WIDGET_INACTIVE != Cy_CapSense_IsWidgetActive(CY_CAPSENSE_BUTTON0_WDGT_ID, &cy_capsense_context))
{
    Cy_GPIO_Write(CYBSP_USER_LED2_PORT, CYBSP_USER_LED2_NUM, CYBSP_LED_ON);
}
else if (CAPSENSE_WIDGET_INACTIVE != Cy_CapSense_IsWidgetActive(CY_CAPSENSE_BUTTON1_WDGT_ID,   &cy_capsense_context))
{
    Cy_GPIO_Write(CYBSP_USER_LED2_PORT, CYBSP_USER_LED2_NUM, CYBSP_LED_ON);
}
else
{
    Cy_GPIO_Write(CYBSP_USER_LED2_PORT, CYBSP_USER_LED2_NUM, CYBSP_LED_OFF);
}
```

CAPSENSE™ low power tuning

9. Build and program the device and observe the green LED turning on when a touch is generated. You may notice that the sensitivity of the sensor is not setup correctly. At this point you may go through the tuning steps from the first lab, or you can apply the following settings to the CSX Button.

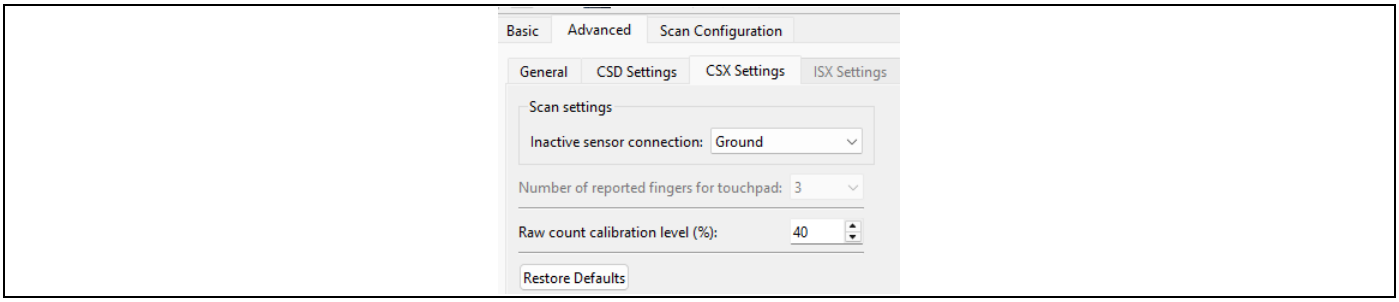

Figure 44 CSX Settings

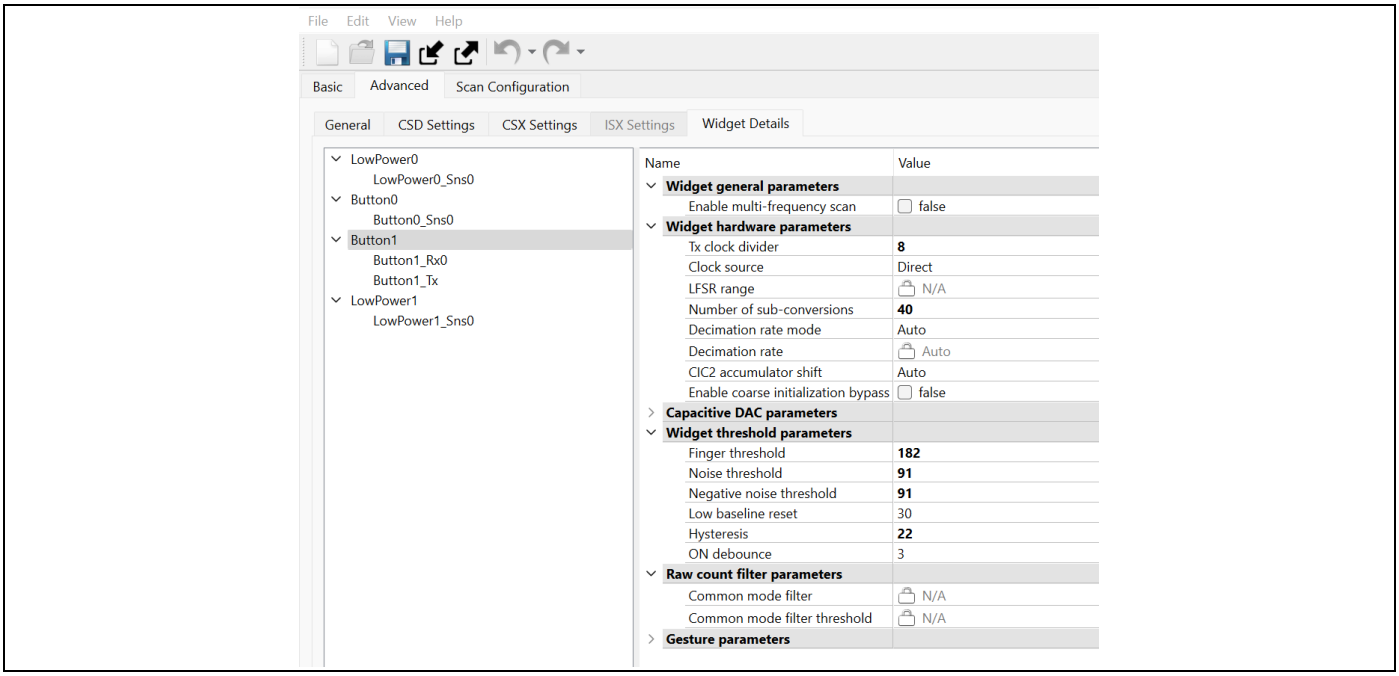

Figure 45 CSX Button Settings

CAPSENSE™ low power tuning

4.4 Tuning a low-power sensor

1. Disable ALR, reduce time for going from Active to WoT mode
    - Change the **ACTIVE_MODE_TIMEOUT_SEC** to 1

```
/* Timeout to move from ACTIVE mode to ALR mode if there is no user activity */
#define ACTIVE_MODE_TIMEOUT_SEC        (1u)
```

    - Update the **ACTIVE_MODE** application power mode state to jump from active mode to wake-on-touch (WOT) mode

```
case ACTIVE_MODE:
   Cy_CapSense_ScanAllSlots(&cy_capsense_context);

   interruptStatus = Cy_SysLib_EnterCriticalSection();

   while (Cy_CapSense_IsBusy(&cy_capsense_context))
   {
      #if ENABLE_PWM_LED
      Cy_SysPm_CpuEnterSleep();
      #else
      Cy_SysPm_CpuEnterDeepSleep();
      #endif

      Cy_SysLib_ExitCriticalSection(interruptStatus);

      /* This is a place where all interrupt handlers will be executed */
      interruptStatus = Cy_SysLib_EnterCriticalSection();
   }

   Cy_SysLib_ExitCriticalSection(interruptStatus);

   #if ENABLE_RUN_TIME_MEASUREMENT
   active_processing_time=0;
   start_runtime_measurement();
   #endif

   Cy_CapSense_ProcessAllWidgets(&cy_capsense_context);

   /* Scan, process and check the status of the all Active mode sensors */
   if(Cy_CapSense_IsAnyWidgetActive(&cy_capsense_context))
   {
      capsense_state_timeout = ACTIVE_MODE_TIMEOUT;
   }
   else
   {
      capsense_state_timeout--;

      if(TIMEOUT_RESET == capsense_state_timeout)
      {
         capsense_state = WOT_MODE;
      }
   }
   #if ENABLE_RUN_TIME_MEASUREMENT
   active_processing_time=stop_runtime_measurement();
   #endif
   break;
   /* End of ACTIVE_MODE */
```

CAPSENSE™ low power tuning

- Update the **WOT_MODE** application power mode state to jump to **ACTIVE_MODE** when the timeout occurs

```
case WOT_MODE :

   Cy_CapSense_ScanAllLpSlots(&cy_capsense_context);
   interruptStatus = Cy_SysLib_EnterCriticalSection();

   while (Cy_CapSense_IsBusy(&cy_capsense_context))
   {
      #if ENABLE_PWM_LED
      Cy_SysPm_CpuEnterSleep();
      #else
      Cy_SysPm_CpuEnterDeepSleep();
      #endif

      Cy_SysLib_ExitCriticalSection(interruptStatus);

      /* This is a place where all interrupt handlers will be executed */
      interruptStatus = Cy_SysLib_EnterCriticalSection();
   }

   Cy_SysLib_ExitCriticalSection(interruptStatus);

   if (Cy_CapSense_IsAnyLpWidgetActive(&cy_capsense_context))
   {
      capsense_state = ACTIVE_MODE;
      capsense_state_timeout = ACTIVE_MODE_TIMEOUT;

      /* Configure the MSCLP wake up timer as per the ACTIVE mode refresh rate */
      Cy_CapSense_ConfigureMsclpTimer(ACTIVE_MODE_TIMER, &cy_capsense_context);
   }
   else
   {
      capsense_state = ACTIVE_MODE;
      capsense_state_timeout = ACTIVE_MODE_TIMEOUT;
      /* Configure the MSCLP wake up timer as per the ACTIVE mode refresh rate */
      Cy_CapSense_ConfigureMsclpTimer(ACTIVE_MODE_TIMER, &cy_capsense_context);
   }

   break;
   /* End of "WAKE_ON_TOUCH_MODE" */
```

2. Set all touch thresholds to maximum (**65535**) to prevent a touch from being detected while tuning the low-power sensor
3. Follow the performance tuning steps
   - Initial parameters set (this is already complete)
   - Update the sense clock frequency
   - Fine tune SNR, power, and refresh rate
   - Tune thresholds
4. Revert changes made in code
   - Change the **ACTIVE_MODE_TIMEOUT_SEC** to 10

```
/* Timeout to move from ACTIVE mode to ALR mode if there is no user activity */
#define ACTIVE_MODE_TIMEOUT_SEC        (10u)
```

CAPSENSE™ low power tuning

- Update the **ACTIVE_MODE** application power mode state to jump from active mode to ALR mode

```
case ACTIVE_MODE:
   Cy_CapSense_ScanAllSlots(&cy_capsense_context);

   interruptStatus = Cy_SysLib_EnterCriticalSection();

   while (Cy_CapSense_IsBusy(&cy_capsense_context))
   {
      #if ENABLE_PWM_LED
      Cy_SysPm_CpuEnterSleep();
      #else
      Cy_SysPm_CpuEnterDeepSleep();
      #endif

      Cy_SysLib_ExitCriticalSection(interruptStatus);

      /* This is a place where all interrupt handlers will be executed */
      interruptStatus = Cy_SysLib_EnterCriticalSection();
   }

   Cy_SysLib_ExitCriticalSection(interruptStatus);

   #if ENABLE_RUN_TIME_MEASUREMENT
   active_processing_time=0;
   start_runtime_measurement();
   #endif

   Cy_CapSense_ProcessAllWidgets(&cy_capsense_context);

   /* Scan, process and check the status of the all Active mode sensors */
   if(Cy_CapSense_IsAnyWidgetActive(&cy_capsense_context))
   {
      capsense_state_timeout = ACTIVE_MODE_TIMEOUT;
   }
   else
   {
      capsense_state_timeout--;

      if(TIMEOUT_RESET == capsense_state_timeout)
      {
         capsense_state = ALR_MODE;
         capsense_state_timeout = ALR_MODE_TIMEOUT;

         /* Configure the MSCLP wake up timer as per the ALR mode refresh rate */
         Cy_CapSense_ConfigureMsclpTimer(ALR_MODE_TIMER, &cy_capsense_context);       }
   }
   #if ENABLE_RUN_TIME_MEASUREMENT
   active_processing_time=stop_runtime_measurement();
   #endif
   break;
   /* End of ACTIVE_MODE */
```

CAPSENSE™ low power tuning

- Update the **WOT_MODE** application power mode state to jump to **ACTIVE_MODE** when the timeout occurs

```
case WOT_MODE :

   Cy_CapSense_ScanAllLpSlots(&cy_capsense_context);
   interruptStatus = Cy_SysLib_EnterCriticalSection();

   while (Cy_CapSense_IsBusy(&cy_capsense_context))
   {
      #if ENABLE_PWM_LED
      Cy_SysPm_CpuEnterSleep();
      #else
      Cy_SysPm_CpuEnterDeepSleep();
      #endif

      Cy_SysLib_ExitCriticalSection(interruptStatus);

      /* This is a place where all interrupt handlers will be executed */
      interruptStatus = Cy_SysLib_EnterCriticalSection();
   }

   Cy_SysLib_ExitCriticalSection(interruptStatus);

   if (Cy_CapSense_IsAnyLpWidgetActive(&cy_capsense_context))
   {
      capsense_state = ACTIVE_MODE;
      capsense_state_timeout = ACTIVE_MODE_TIMEOUT;

      /* Configure the MSCLP wake up timer as per the ACTIVE mode refresh rate */
      Cy_CapSense_ConfigureMsclpTimer(ACTIVE_MODE_TIMER, &cy_capsense_context);
   }
   else
   {
      capsense_state = ALR_MODE;
      capsense_state_timeout = ALR_MODE_TIMEOUT;

      /* Configure the MSCLP wake up timer as per the ALR mode refresh rate */
      Cy_CapSense_ConfigureMsclpTimer(ALR_MODE_TIMER, &cy_capsense_context);
   }

   break;
   /* End of "WAKE_ON_TOUCH_MODE" */
```

5. Follow the steps to measure the current and fill out the table in the Measurements section for the “Two Low Power Sensors” configuration

CAPSENSE™ low power tuning

4.5 Setup single low-power sensor for wake on touch

1. Open the CAPSENSE™ Configurator and remove the **LowPower1** sensor that was added in section 4.3 by selecting the sensor and pressing the **Delete** button

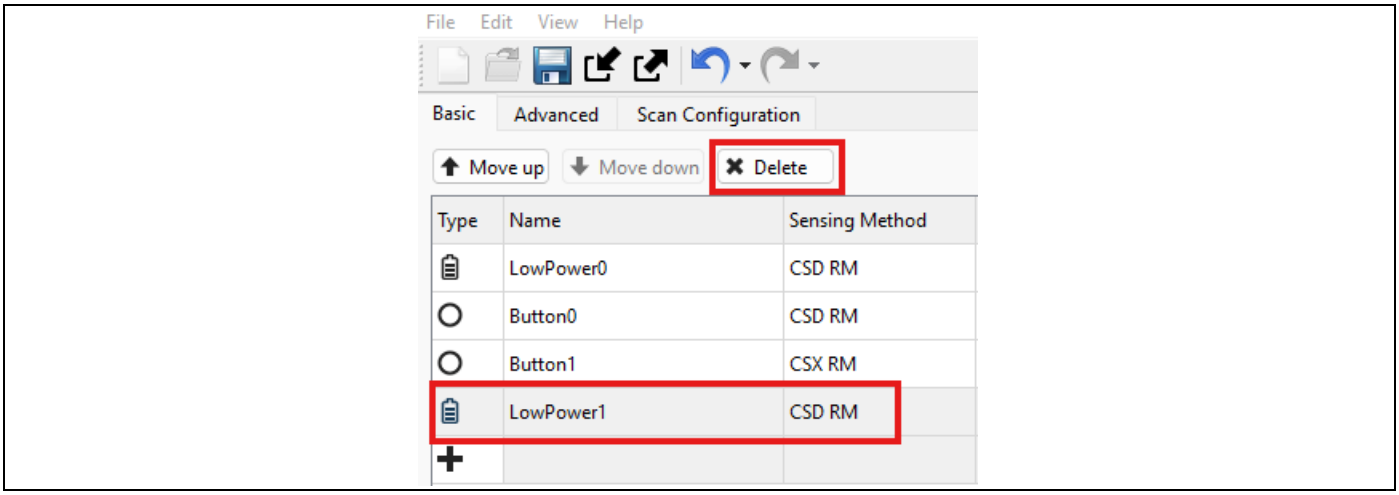

Figure 46 Deleting the LowPower1 sensor

2. Navigate to the **Scan Configuration** tab, then in the **LowPower0** configuration, select the Button0 and Button1 pins in the **Ganged** row to enable ganging of all electrodes for the low-power sensor

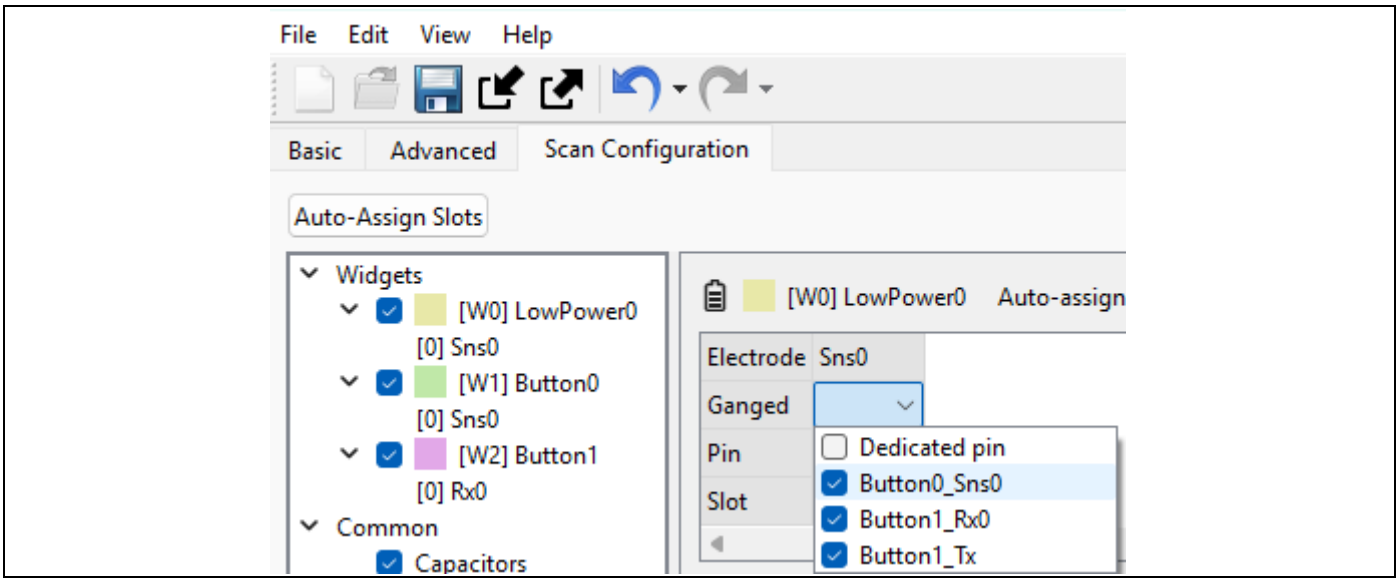

Figure 47 Selecting all CSD and CSX pins to be ganged on the low-power sensor

3. Follow steps to tune the low-power sensor found here
4. Follow the steps to measure the current and fill out the table in the Measurements section for the “One Low Power Sensor” configuration

CAPSENSE™ low power tuning

4.6 Adjust for wake on approach

In some applications it is critical to capture the first touch down of a user input. This is typical for battery powered applications where the user can see the touch surface. In these applications, the user experience often requires that the device detect a “quick tap” or in some cases a “double tap” of the CAPSENSE™ button. To accomplish this, the refresh rate of the low-power sensor can be increased or the touch threshold can be reduced so that a proximity event causes the touch event to be triggered. Increasing the refresh rate will increase the average current consumed. Tuning the low-power sensor for proximity may also cause higher average current if the touch surface is accidentally interacted with.

1. Tune the sensor for proximity by increasing the sensitivity and reducing the touch threshold to 60% of the signal determined from the SNR step while going through the performance tuning

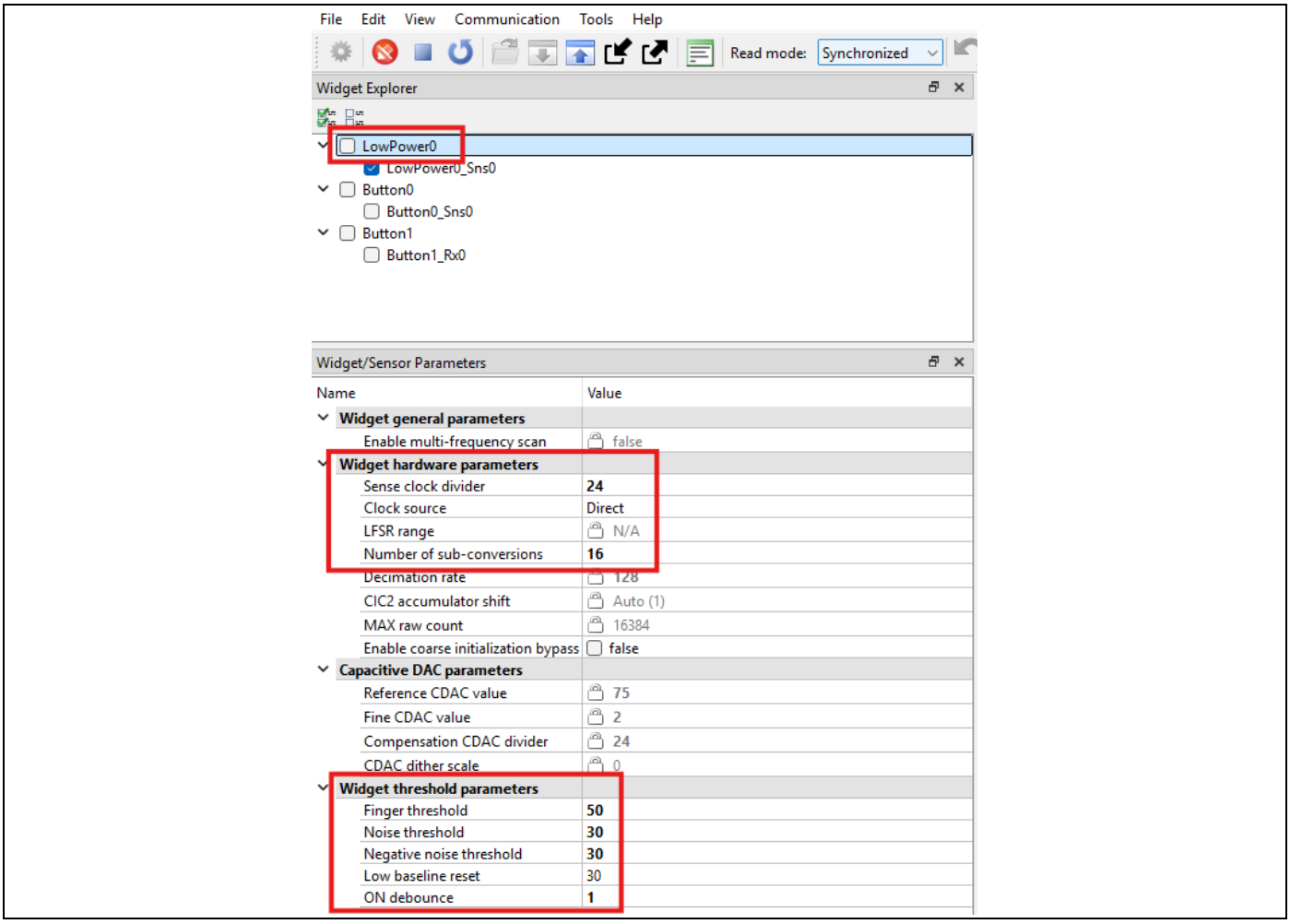

Figure 48 Adjusting the touch threshold on the low-power sensor for proximity

2. Follow the steps to measure the current and fill out the table in the Measurements section for the “One Low Power Sensor; Wake on Approach” configuration

CAPSENSE™ low power tuning

4.7 Measurements

1. Disable the **Debug** port in the **Device Configurator**

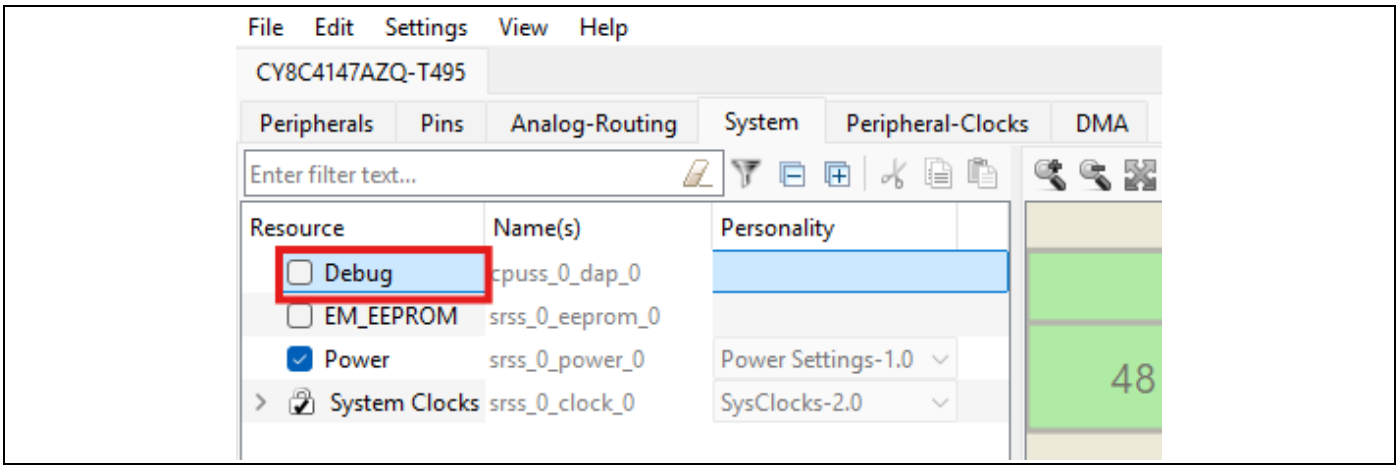

Figure 49 Disable the debug port in the device configurator

2. Set **ENABLE_PWM_LED** to 0

```
/*Enable PWM controlled LEDs*/
#define ENABLE_PWM_LED              (0u)
```

3. Set **ENABLE_TUNER** to 0

```
/* Enable this, if Tuner needs to be enabled */
#define ENABLE_TUNER                (0u)
```

4. Re-program the device then wait until the device is in its Wake-On-Touch application power mode. Measure the current using the J2 jumper and write down the measured current in **Table 6**

Configuration	Expected Current	Measured Current
Two Low Power Sensors	5 µA	
One Low Power Sensor	4.3 µA	
One Low Power Sensor; Wake on Approach	4.5 µA	

Table 6 Current Measurements

CAPSENSE™ low power tuning

4.8 Output

After applying the configuration, test the performance by touching the buttons. If your sensors are tuned correctly, you will observe the touch status go from 0 to 1 in the Status panel of the Graph View tab. The status of the buttons is also indicated by the green LED in the kit. If the wake on touch strategy is used, a clear press is required for the green LED to activate. If the wake on approach strategy is used, quick and double taps should activate the green LED.

4.9 Conclusion

In this exercise we demonstrated how to add a new sensor to a design, how to tune a low-power sensor, and how to implement different low-power sensing strategies. This exercise also showed the steps to get the lowest power out of the device, while still maintaining the required sensing for a given HMI.

References

[1] Infineon Technologies AG: AN85951: PSOC™ 4 and PSOC™ 6 MCU CAPSENSE™ design guide; Available online

Glossary

CAPSENSE™

Infineon® Touch-sensing user interface solution. The industry’s No. 1 solution in sales by 4x over No. 2.

Revision history

Document revision	Date	Description of changes
1.0.0		Initial version created

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.

Edition yyyy-mm-dd
Published by

Infineon Technologies AG
81726 Munich, Germany

Do you have a question about this document?
Email:
erratum@infineon.com
Document reference
User guide number

Important notice
The information given in this document shall in no event be regarded as a guarantee of conditions or characteristics (“Beschaffenheitsgarantie”).

With respect to any examples, hints or any typical values stated herein and/or any information regarding the application of the product, Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind, including without limitation warranties of non-infringement of intellectual property rights of any third party.

In addition, any information given in this document is subject to customer’s compliance with its obligations stated in this document and any applicable legal requirements, norms and standards concerning customer’s products and any use of the product of Infineon Technologies in customer’s applications.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer’s technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

Warnings
Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies’ products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.